Profile-Guided Optimization for Linux Kernel in Data Centers:
From Application-Specific Kernels to Hot-Upgradable Modules

LI, Hui
Student ID: 224040351

A Thesis Submitted in
CSC6032 Advanced Operating System
School of Data Science
The Chinese University of Hong Kong, Shenzhen

April 2026

Abstract

Modern data centers run a small number of large-scale services on each server, but they require high availability, high concurrency, and automated maintenance at a scale that makes manual operating-system tuning impractical. Although Linux is a mature general-purpose operating system, its generality can leave performance on the table for servers that repeatedly execute one dominant workload such as Nginx, Redis, MySQL, or Memcached. Profile-guided optimization (PGO) is a natural response to this gap: instead of optimizing the kernel for all possible execution paths, the compiler can observe the runtime behavior of real services and reorganize hot kernel code around the paths that matter most in production.

This thesis studies profile-guided optimization for the Linux kernel in data centers. It follows the structure of the accompanying presentation and synthesizes four representative lines of work. First, AutoFDO demonstrates that sampling-based feedback-directed optimization can be deployed at warehouse scale in user space without intrusive instrumentation. Second, Tarax extends the PGO idea to kernel space and builds application-specific operating systems by compiling Linux kernels with profiles collected from individual server applications. Third, One Profile Fits All argues that many data center applications share similar kernel control-flow behavior, making a universal Linux kernel profile effective in practice. Fourth, PlugSched shows that a tightly coupled Linux subsystem, the scheduler, can be modularized and updated live with millisecond-level downtime.

The central argument of this thesis is that kernel PGO for data centers is moving through three stages: application-specific optimization, profile generalization, and live modular deployment. Tarax proves the performance potential of specialization, but per-application kernel generation and reboot-based deployment limit operational flexibility. One Profile Fits All reduces the cost of specialization by showing that a merged profile can serve multiple workloads, but the optimized kernel is still deployed as a static whole-kernel artifact. PlugSched addresses the deployment side of the problem by decoupling the scheduler from the rest of the kernel and replacing it online. Building on these insights, the PlugPGO direction explored in the presentation combines PGO with module-level live replacement, aiming to preserve much of the performance benefit of static PGO while avoiding whole-kernel reboot.

Through close reading of the presentation and its cited papers, this thesis identifies the main design trade-off: a production-ready kernel PGO pipeline must simultaneously provide useful speedup, tolerate workload diversity, and support safe online deployment. The conclusion is that future data-center operating systems should integrate automatic profile collection, compiler-guided kernel optimization, and live-update mechanisms into a closed-loop optimization pipeline.

Contents

List of Figures

List of Tables

Chapter 1

Introduction

1.1 Motivation and Data-Center Context

The operating system in a data center is not used in the same way as a desktop operating system. A typical data-center server often runs one dominant service or a small group of co-located services under a controlled deployment environment. The workload is known, the hardware platform is known, and the service-level objective is usually expressed in terms of throughput, tail latency, availability, and operational stability. In this setting, the general-purpose design of Linux is both a strength and a limitation. It is a strength because Linux offers mature device support, a rich ecosystem, and well-tested abstractions. It is a limitation because code paths that are rarely used by a given service still shape compiler decisions, binary layout, branch prediction, cache behavior, and update strategy.

The presentation begins from this observation. In a server that primarily runs a large-scale application, the kernel is not an invisible substrate. System calls, memory management, networking, scheduler activity, virtual file systems, and device drivers are repeatedly exercised by the service. If a web server, database, or key-value store spends a meaningful fraction of cycles in kernel instructions, then kernel layout and branch behavior become application performance issues. The data-center operating system should therefore be seen as a shared performance-critical component rather than a passive compatibility layer. As shown in Figure 1.1, this thesis treats the user application, system-call interface, kernel subsystems, and hardware platform as one performance path rather than as independent layers.

1.2 Why General-Purpose Kernels Are Insufficient

A general-purpose kernel is built to serve a wide range of applications and hardware conditions. Its source code contains broad mechanisms for scheduling, memory reclaim, I/O, networking, virtualization, security, and portability. The compiler cannot assume that one code path is permanently more important than another unless the source code, static heuristics, or profile feedback communicates that information. As a result, a binary compiled without workload feedback may place hot and cold code in the same instruction footprint, make conservative inlining choices, or arrange branches in ways that are reasonable on average but suboptimal for a specific service.

Data-center workloads make this inefficiency especially costly. A small speedup in a service deployed across thousands of machines can translate into fewer servers, less power consumption, or more headroom for traffic bursts. Conversely, a small regression in a hot kernel

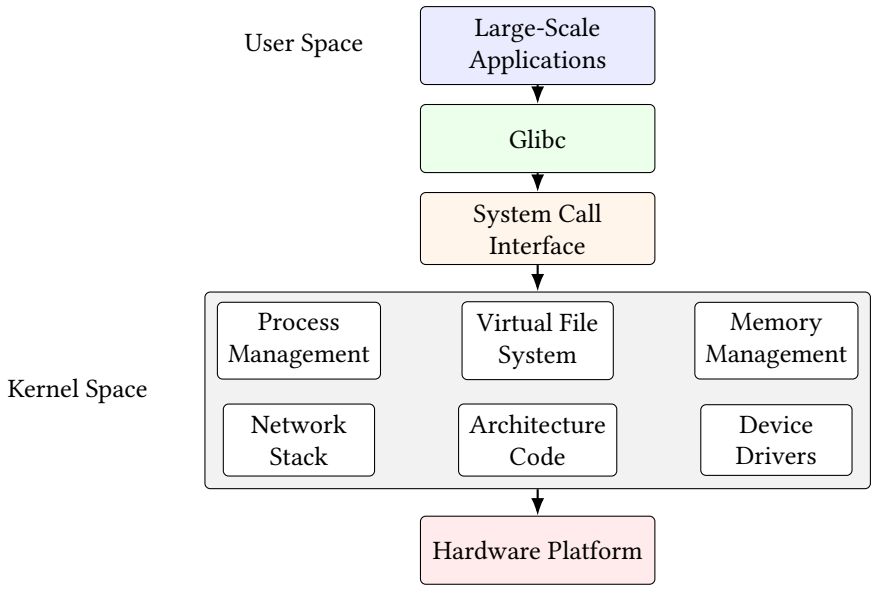


Figure 1.1: Overview of a data-center server architecture.

path can amplify into higher tail latency. Since hardware scaling is expensive and manual kernel tuning does not scale across applications, feedback-directed optimization offers an attractive software-level source of performance.

1.3 Profile-Guided Optimization as an Operating-System Technique

Profile-guided optimization consists of two conceptual phases. In the profile-generation phase, the program is executed under representative workloads and runtime information is collected. In the profile-use phase, the compiler consumes the profile and makes decisions about inlining, branch layout, function ordering, basic-block placement, and other optimizations. In user-space applications, this pattern has been successful because the program can be instrumented or sampled, run under benchmarks, and rebuilt without changing the operating-system deployment model.

Applying PGO to the Linux kernel is more difficult. The kernel does not terminate like a user-space process, so profile collection cannot simply happen on exit. Kernel code has constraints around thread-local storage, instrumentation overhead, bootability, and correctness. The optimized artifact is also operationally sensitive: replacing a whole kernel often requires rebooting or at least a deployment workflow that is incompatible with strict high-availability goals. These constraints motivate the progression studied in this thesis, from application-specific optimized kernels to universal profiles and finally to hot-upgradable optimized modules. The target operating model is the feedback loop shown in Figure 1.2: profile collection, optimized code generation, deployment, and monitoring should reinforce one another.

1.4 Operational Requirements in Data Centers

The data-center setting gives kernel optimization a different meaning from traditional desktop or embedded optimization. On a developer workstation, a faster kernel is valuable, but a reboot or a manually installed experimental image is usually acceptable. In a production cluster,

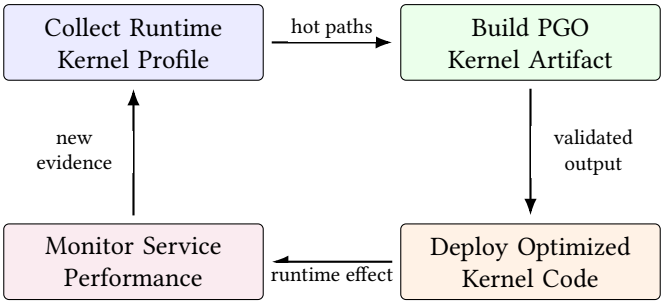


Figure 1.2: Ideal feedback loop for a data-center operating system.

however, the optimization must fit into an operational process. The server must remain available, the update must be reversible, and the performance benefit must be measurable under realistic load. This is why the presentation repeatedly connects performance with automated operation and maintenance. A data-center kernel is not only a compiled artifact; it is part of a fleet-management system.

The first operational requirement is high availability. A service may be replicated across many machines, but each individual reboot still consumes capacity and increases rollout risk. If a PGO kernel requires a maintenance window, then the operator must decide whether the expected speedup is worth the rollout cost. For large fleets, even a small operational friction can prevent frequent optimization. A method that can update a module online, by contrast, can be tested through canary deployment and rolled back quickly if the measured effect is negative.

The second requirement is workload awareness. A generic optimization is not enough when different services stress different kernel paths. A web server dominated by network and socket operations, a key-value store dominated by event handling, and a database dominated by memory and file-system activity may all enter the same Linux kernel, but they do not use the same functions with the same frequencies. PGO is attractive because it can expose these runtime distributions to the compiler. The challenge is to do this without creating an unmanageable number of kernel variants.

The third requirement is automation. The ideal workflow in Figure 1.2 is a feedback loop: run the workload, collect information, recompile or regenerate optimized code, and update the system. If any step requires manual source-code tuning, the approach will not scale to the diversity of modern services. Tarax automates much of the build and profile-use pipeline. One Profile Fits All reduces the burden of profile selection. PlugSched reduces the burden of deployment. PlugPGO is interesting because it tries to combine all three forms of automation.

1.5 Research Questions and Thesis Logic

This thesis is organized around three research questions derived from the presentation. The first question is whether the Linux kernel can be effectively optimized by compiler profile feedback. Tarax answers this question positively by modifying Linux and GCC so that kernel execution profiles can guide compilation. Its results show that kernel PGO is not merely a compiler curiosity; it improves end-to-end application throughput for popular data-center workloads.

The second question is whether profile-guided kernel optimization must be tied to a single application. Application-specific kernels are powerful, but they raise deployment and management costs. One Profile Fits All addresses this question by measuring kernel control-flow similarity across eight data-center applications. Its conclusion is that many services use the

kernel similarly enough that a merged universal profile can preserve most of the benefit of application-specific profiles.

The third question is how optimized kernel code can be deployed. A profile-guided kernel image is not fully useful if the only practical update mechanism is a reboot. PlugSched addresses this question for the Linux scheduler by turning it into a live replaceable module. The PlugPGO direction then asks whether PGO-optimized kernel code can be packaged and replaced through a similar live-update path. These three questions define the thesis argument: performance, generality, and deployability must be solved together.

Thesis contributions. This thesis makes four contributions. First, it reconstructs the motivation for kernel PGO from the data-center operating-system perspective, connecting user-space feedback optimization to kernel-space constraints. Second, it analyzes Tarax as an application-specific kernel PGO system and identifies why its performance gains do not by themselves solve production deployment. Third, it studies One Profile Fits All as a generalization step, showing that similarity in kernel control-flow behavior can reduce the need for one profile per application. Fourth, it treats PlugSched as the missing deployment mechanism and uses it to frame PlugPGO, the presentation's proposed combination of PGO and modular live update.

The methodology is synthetic rather than experimental. No new kernel benchmark is implemented in this thesis. Quantitative claims come from the presentation and the papers cited there: AutoFDO [3], Tarax [14], One Profile Fits All [13], and PlugSched [6]. The goal is to build a coherent thesis-level argument and a reproducible LaTeX artifact from those sources.

1.6 Methodology and Source Selection

The thesis is organized as a paper-style reconstruction of the course presentation rather than as an independent survey of all kernel optimization literature. The presentation defines the main storyline: data-center servers expose repeated kernel behavior; PGO can specialize Linux for that behavior; application-specific kernels are effective but hard to deploy; universal profiles reduce profile-management cost; live update makes optimized kernel code operationally usable; and PlugPGO combines those ideas. The role of the thesis is to expand that storyline into a coherent written argument with enough background, evidence, and design analysis to stand as a course paper.

The figure and table policy follows the same principle. Figures from the presentation are kept when they represent the intended content directly. When a slide figure corresponds to a figure or table in one of the cited papers, the thesis either uses the presentation's version or redraws the result in LaTeX for clarity. The goal is not to introduce unrelated evidence from outside the assigned material. It is to make the presentation's argument readable in paper form. This is why the main evidence comes from AutoFDO, Tarax, One Profile Fits All, and PlugSched, while additional references are used only to explain background ideas such as code layout, application-specific operating systems, and kernel live patching.

The thesis also distinguishes between reported experimental results and proposed system design. AutoFDO, Tarax, One Profile Fits All, and PlugSched each report complete experiments in their own papers. PlugPGO, by contrast, is the method proposed in the presentation. Its UnixBench results are preliminary and are treated as evidence for a deployment direction rather than as proof of a full production system. This distinction is important because the final conclusion should not overstate what has been implemented. The thesis argues that PlugPGO

is a plausible synthesis of the cited systems, while also identifying what must be evaluated before it can be considered production-ready.

This source selection also affects the writing style. Rather than treating every slide as a separate subsection, the revised structure groups related slide ideas into broader arguments. The introduction therefore does not simply list data-center characteristics, PGO motivation, and research questions one by one. It connects them into a single problem statement: a data-center kernel is repeatedly exercised by known services, but the ordinary Linux deployment unit is still a general-purpose whole-kernel image. The rest of the thesis follows this combined performance and deployment problem. Each chapter keeps the presentation's order, but the smaller slide-level ideas are folded into larger prose sections so that the paper reads like a continuous technical argument.

The same principle is used for figures and tables. A figure is placed near the paragraph that explains what question it answers. Architecture diagrams are used to introduce a mechanism, while experimental figures and tables are used after the methodology has been described. This avoids the problem of showing a result before the reader knows why that result matters. It also keeps the thesis close to the format of the reference paper: figures support the surrounding prose rather than interrupting it as isolated slide objects.

1.7 Thesis Organization

Chapter 2 introduces feedback-directed optimization and AutoFDO, establishing the production value of profile collection before moving into kernel space. Chapter 3 studies Tarax and the construction of application-specific Linux kernels. Chapter 4 analyzes the universal-profile argument in One Profile Fits All and asks how much specialization can be shared across services. Chapter 5 explains PlugSched and kernel live update, shifting the discussion from what to optimize to how optimized code can be deployed. Chapter 6 presents PlugPGO as a hot-upgradable PGO direction based on the presentation. Chapter 7 synthesizes the evolution and concludes with open challenges.

Chapter 2

Background and Motivation

2.1 Feedback-Directed Optimization and AutoFDO

Feedback-directed optimization attempts to replace static compiler guesswork with measurements from real execution. A compiler without profile information must infer hot paths from source structure and generic heuristics. A compiler with profile information can instead ask which functions were frequently invoked, which branches were commonly taken, which indirect calls had stable targets, and which loops dominated execution time. That information allows the compiler to optimize for the common case while isolating cold code. For server software, where the common case is often stable across millions of requests, this is an unusually powerful assumption.

Traditional FDO usually relies on instrumentation. The binary is compiled with instrumentation, run under a representative workload, and then rebuilt using the collected profile. This approach can be accurate, but it is awkward at warehouse scale because instrumented binaries are slower and may not be suitable for production traffic. AutoFDO addresses this gap by using sampled hardware performance monitors from production machines and mapping sampled addresses back to source-level profile information.

2.2 AutoFDO in Warehouse-Scale Applications

AutoFDO is an important baseline for this thesis because it shows that feedback-directed optimization can be automated at large scale without requiring every production service to run an instrumented binary. Its pipeline samples hardware performance counters, symbolizes the samples, converts binary-level profiles into source-level profiles, and feeds the resulting profile into GCC. The paper reports that AutoFDO achieves a mean speedup of 10.52% on a set of Google internal applications, which is 84.84% of the traditional instrumentation-based FDO gain for those applications [3]. This is the result highlighted in the presentation as evidence that production-scale feedback optimization is practical. As shown in Figure 2.1, the important systems idea is the automated path from production samples to compiler feedback.

The key idea for kernel PGO is not that AutoFDO can be copied directly into the kernel, but that it establishes a practical pattern: collect profiles from production-like execution, turn them into compiler input, and rebuild optimized binaries automatically. Kernel PGO inherits this ambition while adding stricter constraints. The kernel cannot be treated as a normal process, and the cost of a bad optimization is not merely application failure but potential system instability.

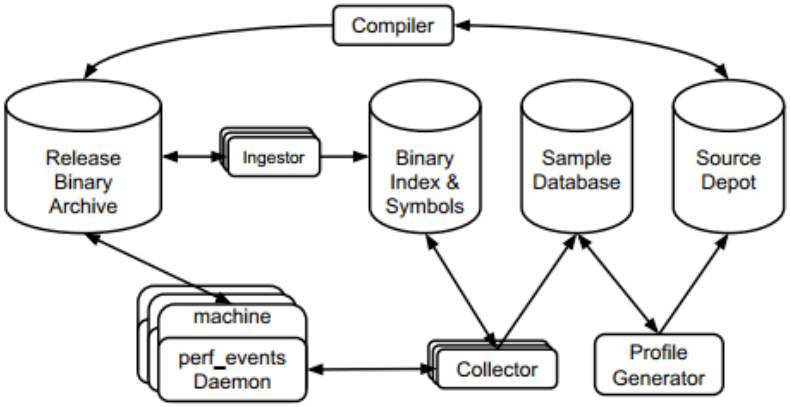


Figure 2.1: AutoFDO system workflow.

2.3 From Binary Samples to Source Profiles

AutoFDO must bridge the gap between binary-level events and source-level compiler decisions. Hardware samples identify instruction addresses. The compiler, however, needs feedback attached to functions, lines, basic blocks, call edges, and inline contexts. The AutoFDO paper therefore relies on debug information and extended source locations to map sampled instructions to source constructs. This mapping is imperfect, especially when aggressive optimization has transformed loops or inlined functions, but it is good enough to recover much of the benefit of instrumentation-based FDO. The mapping problem is illustrated in Figure 2.2, following the source-profile construction described by Chen et al. [3].

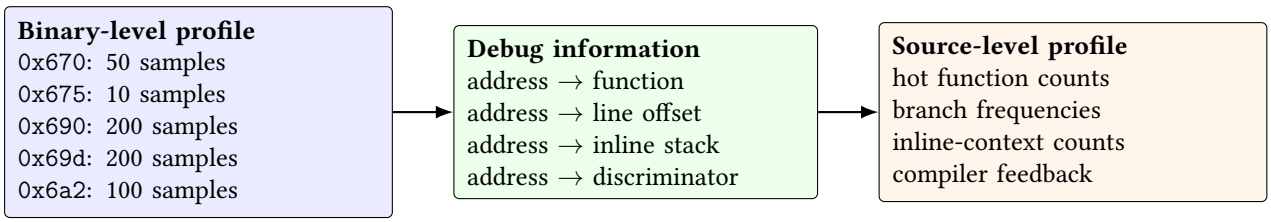


Figure 2.2: Binary-level samples are converted into source-level feedback using debug information and extended source locations.

The quantitative evidence for this claim is summarized in Tables 2.1 and 2.2. In Chen et al. [3], the Google internal application results show that AutoFDO recovers most of the traditional FDO benefit, while the SPEC results show that sampled feedback is also competitive on standard CPU benchmarks.

2.4 Systems Context before Kernel PGO

The PGO idea used in AutoFDO and Tarax has a long compiler history. A classic form of profile-guided optimization is profile-guided code positioning, where the compiler or binary optimizer places frequently executed basic blocks and functions close together so that instruction-cache locality improves [10]. This idea is directly relevant to the kernel because the Linux text segment contains many subsystems, many error paths, and many rarely executed branches. Without profile information, hot and cold paths can be interleaved in ways that enlarge the active instruction footprint.

Table 2.1: AutoFDO and instrumentation-based FDO on Google internal applications.

Application	FDO	AutoFDO	AutoFDO/FDO
server	17.61%	15.89%	90.23%
graph1	14.68%	14.04%	95.65%
graph2	7.16%	6.27%	87.50%
machine learning1	8.92%	8.46%	94.85%
machine learning2	7.09%	6.60%	93.06%
encoder	8.63%	3.31%	38.37%
protobuf	16.96%	14.40%	84.94%
artificial intelligence1	10.12%	10.12%	100.00%
artificial intelligence2	13.24%	11.33%	85.61%
data mining	20.48%	15.54%	75.86%
Mean	12.40%	10.52%	84.84%

Table 2.2: AutoFDO and FDO on selected SPEC CPU2006 integer benchmarks.

Benchmark	FDO	AutoFDO	AutoFDO/FDO
400.perlbench	15.27%	14.99%	98.17%
403.gcc	7.73%	7.52%	97.28%
445.gobmk	3.67%	3.23%	88.01%
458.sjeng	6.19%	6.03%	97.42%
473.astar	8.86%	10.12%	114.20%
483.xalancbmk	14.44%	11.98%	82.96%
Mean over SPEC integer suite	4.40%	4.87%	112.33%

Modern data-center binary optimizers such as BOLT revisit the same principle at binary level: observe execution, identify the hot control-flow graph, and rearrange code after linking [8]. The presentation does not focus on binary post-link optimization, but the same motivation appears in One Profile Fits All. If front-end stalls caused by I-cache and I-TLB misses are expensive, then code placement is not a secondary issue. It becomes a system-level performance concern. The kernel is an especially attractive target because the same optimized kernel can serve many processes on the machine.

There is also an important distinction between user-space binary optimization and kernel optimization. User-space services can often be restarted independently. Their optimized binaries can be deployed behind load balancers or process supervisors. A kernel update, by contrast, affects the whole machine. The cost of failure and the cost of rollback are higher. This is why the thesis does not stop at AutoFDO: the compiler technique must be paired with an operating-system update mechanism.

Application-specific operating-system lineage. Tarax belongs to a broader tradition of application-specific operating systems. Earlier systems such as Exokernel argued that applications could achieve better performance if the operating system exposed low-level resources and allowed application-level resource management [4]. Later library operating systems and unikernels showed that a service-specific OS image could reduce footprint and remove unnecessary generality in cloud environments [7]. Data-plane systems such as Arrakis and IX similarly targeted high throughput and low latency by restructuring the boundary between applications and kernel services [2, 9].

These systems are useful background because they show why specialization is attractive. If the operating system can be tailored to a workload, the service may avoid generic layers, reduce context switching, and improve locality. However, they also show why manual specialization is difficult to adopt broadly. A data center can run thousands of services and kernel configurations. Rewriting the OS for each service is not realistic. Tarax's compiler-based approach is therefore conservative: it keeps Linux as the common source base and uses PGO as the specialization mechanism. This preserves much of the compatibility of Linux while still pursuing the performance logic of application-specific systems.

Live update and the deployment side of optimization. The final background thread is live update. Function-level kernel patching systems such as Ksplice and kpatch show that some kernel updates can be applied without rebooting [1, 5, 11]. These systems are important because they change the operational model of kernel maintenance: a fix can be applied to a running machine while services continue. However, function-level patching is not expressive enough for large component changes. The PlugSched paper explicitly compares this design point with component-level scheduler update.

The ghOSt scheduler demonstrates another way to make scheduling more flexible by delegating policy to user space [12]. This improves programmability, but it also changes the scheduler architecture and may introduce context-switch overhead or deployment constraints that PlugSched wants to avoid. PlugSched's approach is different: keep scheduler execution in the kernel, but modularize it so that a new scheduler can be loaded and redirected online. For PlugPGO, this distinction is central. The goal is not to move optimized kernel logic out of the kernel, but to update optimized kernel logic safely inside the kernel deployment model.

2.5 From Background to Research Lineage

AutoFDO is not a kernel optimizer, but it resolves a deployment objection that would otherwise weaken the case for PGO. If profile generation required every production binary to be rebuilt in a slow instrumented mode, then feedback-directed optimization would remain a specialized benchmarking technique. AutoFDO shows that sampling can make profile collection compatible with large production fleets. This compatibility is the part that matters for data-center kernels: profile collection must be routine, low-overhead, and automatable.

At the same time, AutoFDO also exposes the central weakness of sampled feedback. The compiler sees an approximation of execution rather than exact instrumentation counts. The mapping from binary samples to source locations depends on debug information, and aggressive optimization can distort that mapping. Kernel PGO inherits the same problem and adds new ones: kernel profiles include interrupt context, scheduler interactions, and subsystem effects that are harder to isolate than a single user-space binary. Therefore, AutoFDO motivates kernel PGO, but Tarax is needed to make the idea concrete inside the Linux kernel.

Kernel-space challenges. The presentation identifies two obstacles that make kernel PGO harder than user-space PGO. The first is the lack of convenient sampling and profiling tools inside kernel mode. Instrumentation must not break boot, scheduling, interrupt handling, or low-level architecture assumptions. User-space facilities such as thread-local storage cannot always be used safely. Profile data must be collected while the kernel continues running rather than when a program exits. The second obstacle is update strategy. In a data center, replacing the whole operating system kernel through reboot conflicts with high availability and automated maintenance goals.

These constraints explain why a pure compiler solution is insufficient. A compiler can pro-

duce an optimized kernel image, but production adoption also requires a deployment mechanism. The rest of the thesis follows that logic. Tarax answers the compiler question: can Linux be optimized with profile feedback for a particular workload? One Profile Fits All answers the generalization question: can one profile work across many data-center applications? PlugSched answers the deployment question: can a kernel subsystem be updated live as a module? PlugPGO asks whether those answers can be combined.

2.6 Relationship among the Four Core Papers

The four papers used in this thesis should not be read as isolated techniques. They form an argument about how a data-center kernel optimization pipeline could be built. AutoFDO is the starting point because it establishes a production-friendly way to collect profile information. The key insight is not merely that feedback-directed optimization improves performance. Traditional FDO had already shown that. The important point is that profile collection can be separated from expensive instrumentation and can rely on hardware sampling. This makes profiling compatible with warehouse-scale services where operators cannot afford to run slow instrumented binaries for every workload.

Tarax takes the next step by moving from application binaries to the Linux kernel. Its contribution is to show that the kernel can be treated as an optimization target rather than only as a fixed execution environment. This is a conceptual shift. In ordinary deployment, an application team tunes the application and accepts the kernel as part of the platform. Tarax says that, when the server runs a known application, the platform itself can be compiled around the application's kernel behavior. The method is still conservative because it keeps the Linux source base and the normal kernel programming model. The specialization comes from compiler feedback, not from rewriting the kernel by hand.

One Profile Fits All then asks whether Tarax's strongest assumption is necessary. Tarax assumes that each workload should have its own optimized kernel. This is reasonable for maximum performance, but difficult for fleet management. A large data center may run many services, many kernel versions, and many hardware generations. Maintaining one optimized kernel per workload can quickly become an operational burden. One Profile Fits All observes that many data-center services enter the same Linux kernel and use similar hot functions and branches. Therefore, a merged profile can capture enough of the common behavior to provide useful speedup. In the overall argument of this thesis, this paper reduces the number of optimized artifacts that operators need to manage.

PlugSched addresses a different axis. It does not try to improve compiler optimization. Instead, it asks how an already changed kernel component can be placed into a running system safely. This matters because profile-guided optimization is only useful in production if the optimized code can be deployed repeatedly. A one-time offline benchmark result is less valuable than a workflow that allows operators to collect new profiles, compile new code, deploy it, measure the result, and roll back if needed. PlugSched supplies the mechanism and vocabulary for that workflow: component boundaries, redirection, state handling, downtime, rollback, and memory safety.

The resulting relationship is therefore cumulative. AutoFDO contributes scalable profiling, Tarax contributes kernel PGO feasibility, One Profile Fits All contributes profile reuse, and PlugSched contributes online deployment. PlugPGO is useful because it combines the missing parts. It is not a replacement for the earlier work. Rather, it is a deployment-oriented synthesis: profile information should guide kernel code generation, but the optimized unit

should be small enough to update online.

This relationship also defines how the later chapters should be evaluated. A useful kernel PGO system cannot be judged only by peak benchmark speedup. If the profile is expensive to collect, the system will not be used often. If the optimized artifact is too specific, the system will create too many variants for a production fleet. If the update requires reboot, the system will not support fast feedback. The cited papers each solve one part of this chain, and their limitations point naturally to the next part. This is why the thesis treats PGO as a pipeline rather than as a compiler flag.

The background chapter therefore prepares three evaluation dimensions used throughout the paper. The first is profiling cost: how the system observes real execution and whether that observation can be repeated in production. The second is optimization scope: how much of the kernel is optimized and whether the profile is application-specific, universal, or hybrid. The third is deployment granularity: whether the output is a whole kernel, a component, or a hot-upgradable module. These dimensions make the comparison among AutoFDO, Tarax, One Profile Fits All, PlugSched, and PlugPGO more systematic.

Chapter 3

Application-Specific OS via Tarax

This chapter first reconstructs the original Tarax paper as a system for building application-specific Linux kernels. The first three sections therefore follow the paper's own logic: motivation, architecture, toolchain support, and evaluation. Only after that does the chapter shift to the thesis-level interpretation: what Tarax teaches us about the cost of per-application kernels and why a smaller deployment unit becomes attractive.

3.1 Original Tarax Problem: Application-Specific Linux

Application-specific operating systems are attractive in data centers because the workload is often stable and known. If a server runs Apache, the operating system can reasonably favor Apache's kernel behavior over paths that only matter to unrelated workloads. Prior application-specific OS designs often required manual redesign or a library-OS style reimplementation. Tarax proposes a less invasive alternative: keep Linux as the common source base, collect application-specific profiles, and let compiler optimizations specialize the kernel binary [14]. As shown in Figure 3.1, the same Linux source tree can therefore produce different optimized kernels for different dominant applications.

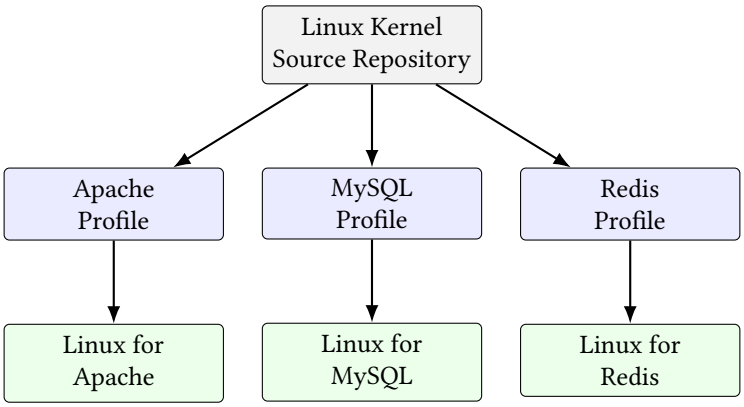


Figure 3.1: General idea of application-specific operating systems based on a common Linux source tree.

Tarax is important because it shows that the application-specific OS idea does not require discarding the Linux ecosystem. The same source repository can be compiled into different optimized kernels for different services. This is a conservative and practical choice: the kernel interfaces remain familiar, application code is not rewritten, and the specialization is carried primarily by compiler feedback rather than manual OS redesign.

This is the main reason Tarax fits the data-center setting. Operators usually prefer optimization techniques that preserve the existing Linux programming model. A unikernel or library operating system can remove more generality, but it also changes debugging, deployment, and compatibility assumptions. Tarax keeps the compatibility base stable and lets profiles guide the compiler toward the hot paths of a service. In this sense, the system is application-specific in performance behavior, but conservative in software-engineering posture.

3.2 Original Tarax System Design

The Tarax pipeline follows the basic PGO structure but adapts each phase to the Linux kernel. First, an instrumented kernel is built with profiling support. Second, the target application runs on that kernel, generating profile data that reflects real kernel behavior under the application workload. Third, a utility program collects and converts the profile data. Fourth, the Linux kernel is recompiled with the collected profile, producing an application-specific optimized kernel image. As shown in Figure 3.2, this pipeline makes profile generation and profile use separate build phases, which is why Tarax must modify both Linux and GCC [14].

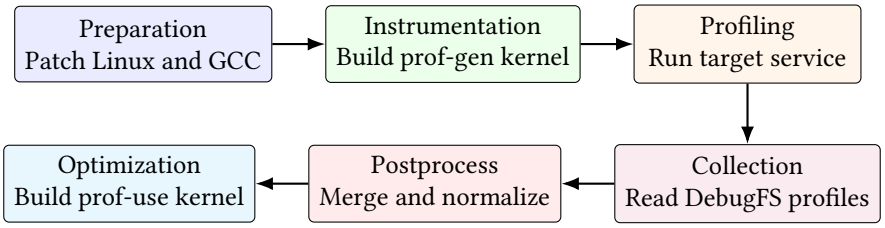


Figure 3.2: Building and optimization workflow in Tarax.

The workflow is mostly automated, but it still contains two operationally meaningful manual steps: booting the instrumented kernel and running the representative target application. These steps are acceptable for research and controlled deployments, but they also foreshadow the production pipeline problem. A data center can automate benchmark or canary runs, yet it must still decide when the profile is representative, when the kernel should be rebuilt, and how the optimized image should be deployed.

The workflow also makes clear why profiles are not interchangeable artifacts. A profile is tied to a kernel version, a compiler version, a kernel configuration, and a workload. If any of these changes, the optimized output may no longer represent the intended execution. Tarax demonstrates the build flow, while later chapters ask how to manage the resulting artifacts when a fleet contains many applications and frequent kernel updates.

System architecture and kernel modifications. Tarax modifies Linux, GCC, and profile-processing tools. On the kernel side, it extends the `gcov` subsystem so that profiling information can be collected from a kernel that does not exit. It also addresses the problem of value profiling and disables assumptions such as thread-local storage that are unsafe or unavailable in kernel mode. On the compiler side, it modifies GCC behavior to handle kernel-specific optimization problems and selective size optimization. These modifications are not incidental engineering details: they are the reason a standard user-space PGO workflow can be made to work for Linux. The component relationship is shown in Figure 3.3, where the kernel-side profile collector and compiler-side profile consumer form one toolchain [14].

The architecture also illustrates the boundary between general PGO and kernel-specific PGO. General PGO provides the conceptual compiler mechanism. Tarax contributes the kernel plumbing needed to collect profile data, the compiler changes needed to avoid incorrect

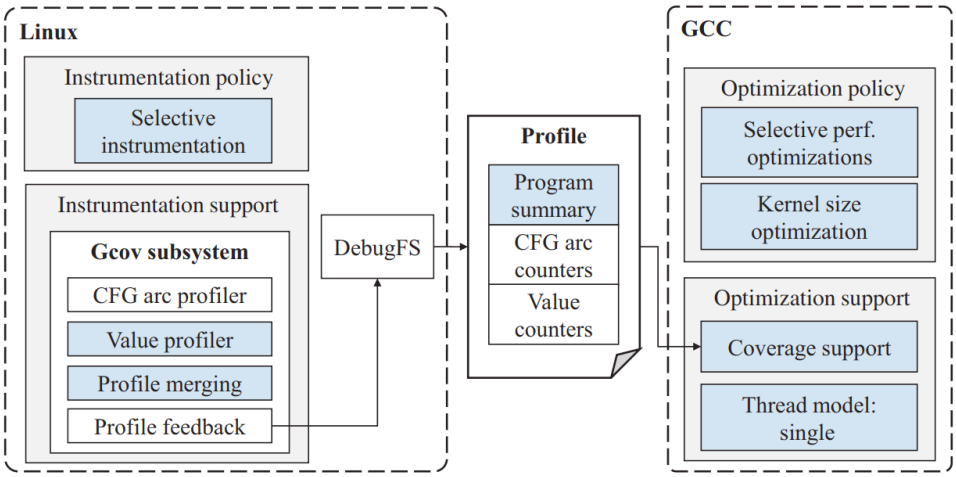


Figure 3.3: System architecture of Tarax.

optimization, and the automation needed to build multiple Linux kernels from a shared source tree. Without those parts, the idea of "just use PGO on the kernel" is incomplete.

This chain of components also explains why the chapter places architecture before evaluation. The performance results are meaningful only because the preceding architecture preserves kernel correctness while making profiles available to the compiler. The paper's engineering contribution is therefore not simply that an optimized kernel is faster, but that the profiling and build pipeline can be made complete enough to produce bootable optimized Linux kernels for real services.

Kernel instrumentation challenges. Tarax's implementation details are important because the kernel is not a normal instrumented program. In a user-space PGO workflow, instrumentation counters can be stored in process memory and written out when the process exits. The Linux kernel does not exit during normal operation, and it serves many processes, interrupts, and kernel threads. Tarax therefore has to integrate with the kernel's profiling facilities and expose profile data through mechanisms that can be collected while the system is running.

The presentation highlights value profiling, selective instrumentation, and disabling thread-local storage. Value profiling matters because GCC can use runtime values to make optimization decisions, but the standard implementation assumes user-space runtime support. Selective instrumentation matters because instrumenting every kernel path can inflate overhead and code size. Disabling thread-local storage matters because TLS assumptions that are natural in user space can be unsafe in kernel mode. These are not minor portability issues; they are exactly the places where a user-space compiler workflow collides with kernel execution semantics.

Tarax also has to decide when a profile is representative. A server application often has a warm-up phase, a steady-state phase, and sometimes background maintenance activity. Collecting a profile too early may overrepresent initialization paths. Collecting a profile under an unrealistic benchmark may optimize the wrong code. The paper's evaluation therefore uses workload-specific benchmark tools for each application. This follows the same principle as AutoFDO: the profile should reflect the execution that matters in production, not merely a synthetic path that is easy to run.

The instrumentation discussion leads naturally to the compiler discussion. If the profile is expensive or intrusive to collect, then the compiler receives a distorted signal. If the compiler cannot safely consume that signal, then the profile is useless. Tarax must therefore solve

both sides together: profiling must be kernel-aware, and optimization must be constrained by kernel correctness.

Compiler modifications and optimization safety. The Tarax paper also modifies GCC because enabling profile feedback can activate optimizations that are unsafe for some kernel functions. The kernel contains code that is compiled under strict assumptions about calling conventions, section placement, inline assembly, and architecture-specific behavior. An optimization that is profitable for a normal application may cause a kernel build failure or even generate incorrect code. Tarax therefore treats compiler integration as part of the system design.

This safety point is part of Tarax's original contribution. The paper does not treat PGO as an isolated compiler flag; it treats it as a constrained kernel build mode. An optimized kernel must remain bootable, preserve low-level calling and section-placement assumptions, and avoid transformations that are profitable for ordinary applications but unsafe for kernel code. Tarax solves this problem at whole-kernel build time by constraining GCC behavior and by treating toolchain changes as part of the operating system design.

The most practical lesson is that kernel PGO is not one feature flag. It is a toolchain pipeline. Linux configuration, compiler options, instrumentation code, profile collection, profile postprocessing, and deployment must agree with one another. A failure in any stage can erase the benefit of the compiler optimization. This is why the presentation frames kernel PGO as an operating-system problem rather than only a compiler problem.

The experimental setting used to validate that toolchain is summarized in Tables 3.1 and 3.2. These tables follow Yuan et al. [14] and matter because kernel PGO results depend on the kernel version, compiler version, hardware platform, and application versions.

Table 3.1: Tarax experimental environment.

Type	Parameter
Processor	Intel Core i7-4770
Memory	32 GB DDR3 1600 MHz
Network	10 Gbps LAN
Kernel	Linux 4.1.2
Kernel compiler	GCC 5.1.1
Operating system	Debian sid amd64
File system	tmpfs

Table 3.2: Application versions used in the Tarax evaluation.

Application	Version
Apache	2.4.23
Nginx	1.10.2
MySQL	5.6.25
PostgreSQL	9.3.9
Redis	3.0.2
Memcached	1.4.21

3.3 Original Evaluation: Workloads, Methodology, and Performance

Tarax evaluates six popular server applications: Apache, Nginx, MySQL, PostgreSQL, Redis, and Memcached. The main result is that every target application improves when it runs on its corresponding optimized kernel, with an 8.8% geometric-mean improvement and a maximum improvement of 16.9% for Nginx. These numbers are large enough to matter in data centers and comparable to user-space PGO gains. As shown in Table 3.3, the speedup is workload dependent, which is the empirical basis for Tarax's application-specific design [14].

Table 3.3: Application performance on vanilla Linux and Tarax-optimized kernels.

Application metric	Vanilla mean	Tarax mean	Improvement
Apache (requests/s)	61,843	69,186	11.9%
Nginx (requests/s)	255,397	298,443	16.9%
MySQL (trans/min)	70,499	74,489	5.7%
PostgreSQL (trans/min)	80,943	83,194	2.8%
Redis (operations/s)	367,807	396,407	7.8%
Memcached (operations/s)	427,715	464,129	8.5%
Average (geomean)	–	–	8.8%

The evaluation is also revealing because the speedups are not uniform. Web-serving workloads benefit more than PostgreSQL in the reported setup. This does not weaken the argument for kernel PGO; rather, it reinforces the application-specific premise. Different applications stress different kernel paths, and the compiler's best layout for one service need not be best for another. The very existence of this variation is what motivates both Tarax and the later One Profile Fits All question.

Benchmarking methodology and representativeness. Tarax's evaluation methodology is worth discussing in detail because profile-guided optimization depends strongly on representativeness. The paper does not use a single generic benchmark for all applications. Instead, each service is evaluated with a tool that matches its normal usage model: web servers are tested with HTTP request generators, databases with transaction-processing benchmarks, and key-value stores with request mixes that exercise get and set operations. This matters because the kernel profile is shaped by the application interface. A database transaction benchmark and a web-serving benchmark may both generate system calls, but their timing, memory behavior, and I/O patterns differ.

The benchmark design also explains why Tarax uses both a test machine and a client machine connected by high-speed network. If the client becomes the bottleneck, the server-side kernel profile will not represent the service's real hot paths. If disk I/O variance dominates the measurement, the compiler effect may be hidden by storage noise. The paper therefore uses tmpfs to reduce disk uncertainty. These choices show that kernel PGO evaluation is not only about compiling a new kernel; it is about constructing a measurement environment where kernel behavior can be observed clearly.

A second representativeness issue is run-to-run stability. Tarax reports standard deviations for the application results, which helps distinguish stable optimization effects from noise. This is especially important for operating-system experiments because small changes in scheduling, interrupt timing, or cache state can affect measured throughput. The fact that Tarax reports consistent positive improvements across all six applications supports the con-

clusion that the profile feedback is affecting real kernel behavior rather than producing a one-off benchmark artifact.

Finally, the benchmark methodology connects the paper's system construction to its performance claims. Tarax does not merely show that a kernel can be rebuilt with profile feedback; it shows that the resulting binary should be judged under representative service-level metrics. Profiles must be collected under realistic load, and the optimized kernel must be validated against the application's own throughput metric. This discipline is what makes the reported speedup credible rather than a compiler demonstration in isolation.

Where Tarax's speedup comes from. Tarax attributes its gains mainly to better kernel code layout and branch behavior. The paper reports that kernel I-cache miss rates are reduced more significantly than application I-cache miss rates, which is exactly what a kernel PGO system should achieve. The optimized kernel also reduces taken conditional branches for several workloads, which can improve fall-through locality and instruction-cache utilization. These effects show that the speedup is not merely a benchmarking artifact. The profile changes the compiler's understanding of which kernel paths dominate execution.

This point is important for interpreting Tarax's scope. Some benefits come from whole-kernel compilation and global layout decisions, but the paper also points to local effects such as branch direction, hot basic-block placement, and instruction-cache behavior inside frequently executed kernel paths. The distinction matters later because local effects are more likely to survive if the deployment unit is smaller than a whole kernel image.

Application specificity and cross evaluation. One of the most important questions in Tarax is whether the optimized kernels are truly application-specific. If an Nginx-optimized kernel also improved every other service by the same amount, the result would be better interpreted as a generic compiler configuration. Tarax therefore performs cross evaluation: each application is run not only on its own optimized kernel, but also on kernels optimized for other applications. The purpose is to test whether the profile captures workload-specific kernel behavior.

The qualitative conclusion is that specialization matters. Web servers, databases, and key-value stores exercise different mixes of network, memory, timer, file-system, and scheduler code. A kernel built with the Redis profile is not necessarily the best kernel for MySQL. This supports the presentation's motivation that a data-center OS can be made more efficient when it is customized for the dominant application. It also reveals the management problem: if every workload needs its own kernel image, the number of variants grows quickly.

This cross-evaluation result is the direct predecessor of One Profile Fits All. Tarax shows that profiles are specific enough to matter. One Profile Fits All asks whether they are also similar enough to merge. The two claims may appear contradictory, but they are actually compatible. Individual applications can have different best profiles while still sharing a large common kernel hot path. A universal profile may not be perfectly optimal for every service, yet it can be operationally preferable if it captures most of the common behavior and avoids maintaining many kernel variants.

Workload, hardware, and kernel-version sensitivity. The Tarax paper studies sensitivity along three dimensions that matter for real deployment. The first is workload sensitivity. Even within a single application such as Nginx or Memcached, changing the request mix can change which kernel paths are hot. For example, static and dynamic web requests may stress different parts of the network stack and file-system path. If the profile is collected under one request distribution but the production workload changes, the optimized kernel may lose part of its benefit.

The second dimension is hardware sensitivity. Kernel code layout affects instruction cache

and branch behavior, but the exact cost of misses depends on the processor. Cache size, branch predictor behavior, memory hierarchy, and NUMA layout can all change how valuable a particular code arrangement is. Tarax reports that the optimization remains useful across hardware platforms, which is encouraging, but the result also suggests that profile pipelines should be aware of the target machine class.

The third dimension is kernel-version sensitivity. Linux evolves quickly, and the source code around a hot path can change between versions. A profile collected for one kernel version may not map perfectly to another. Tarax evaluates multiple Linux versions and observes that the average improvement remains positive. For a production system, this means profile-guided kernel optimization should be integrated into the normal kernel upgrade pipeline rather than treated as a one-time tuning activity.

3.4 My Analysis: Operational Cost and PlugPGO Implications

The preceding sections describe Tarax as the original paper presents it: a Linux-compatible way to specialize the kernel for a known server workload. From this point, the discussion shifts from the paper's demonstrated mechanism to my interpretation of its architectural implications for data-center deployment.

Tarax's main strength is that it converts a conceptual opportunity into an implemented system. It demonstrates that Linux can be instrumented, profiled, recompiled, and measurably accelerated for real server applications. It also shows that compiler-driven specialization is less invasive than redesigning the operating system manually. For production teams, this matters because a Linux-compatible workflow is much easier to adopt than a custom OS stack.

The limitation is deployment complexity. Tarax produces optimized whole-kernel images. Each target application may require its own profile and its own optimized kernel. Even if the build process is automated, switching the running server to the optimized kernel is not a millisecond-level online operation. This limitation is the bridge to the next chapters: Can one profile serve many applications, and can optimized kernel components be deployed without reboot?

Operational cost of per-application kernels. The operational cost of Tarax appears only indirectly in performance charts, but it is central to the thesis. A per-application kernel is easy to understand in an experiment: run Apache, collect a profile, build an Apache-oriented kernel, and compare performance. In a data center, the same idea becomes a fleet-management problem. Operators must decide which services deserve separate profiles, how often profiles should be refreshed, which kernel source version each profile belongs to, and how optimized images should be rolled out across machines. These tasks are not impossible, but they change PGO from a compiler option into a full operating-system maintenance process.

Versioning is the first cost. A profile is meaningful only with respect to a particular kernel source tree, configuration, compiler, and workload. If the kernel is patched for security, the optimized image may need to be rebuilt. If the application changes its request mix, the profile may become stale. If hardware changes, the hot paths may remain similar but the instruction-cache and branch-prediction effects may shift. Therefore, a production Tarax-style system would need profile metadata, artifact tracking, and automated rebuild policy.

Deployment is the second cost. Rebooting into a new optimized kernel can be acceptable for scheduled maintenance, but it is a poor match for frequent optimization. Data-center operators normally prefer canary rollout, incremental expansion, and quick rollback. A whole-

kernel PGO artifact makes each optimization more expensive to test. If a new profile improves throughput for one workload but hurts another co-located task, rollback may require booting back into the previous kernel. This discourages experimentation even when profile feedback might be valuable.

The third cost is fragmentation. If every service has its own optimized kernel, the fleet may contain many kernel variants. More variants can mean more test combinations, more debugging complexity, and more uncertainty when diagnosing production incidents. A small performance gain may not justify a large increase in operational state. This does not make Tarax impractical, but it explains why the next step in the presentation is One Profile Fits All. Reducing the number of profile variants is not merely convenient; it is part of making kernel PGO maintainable.

This cost analysis also clarifies the value of PlugPGO. Module-level update does not remove the need for profiles, versioning, or validation, but it reduces the deployment unit. Instead of treating every optimized profile as a new kernel image, the system can treat it as an update to a selected hot component. Smaller deployment units make profile experimentation less disruptive and make rollback easier. In this sense, PlugPGO is not a contradiction of Tarax. It is an attempt to preserve Tarax's performance logic while lowering its operational cost.

Tarax is therefore best understood as a proof of performance potential rather than as the final deployment architecture. Its experiments answer the question of whether kernel PGO is worth pursuing at all. The answer is yes: the Linux kernel contains enough workload dependent behavior that compiler feedback can improve real server applications. Once that point is established, the research problem changes. The next question is no longer whether specialization can work, but how specialization should be packaged so that it can survive the operational constraints of a data center.

This distinction is important for interpreting the figures in this chapter. The workflow diagrams show how Tarax collects and uses profile information, while the performance figures show that the optimized kernels improve throughput across applications and hardware platforms. Together, they support a two-part conclusion. First, application specificity is a meaningful source of performance because different services stress different kernel paths. Second, the optimization mechanism must be made less disruptive before it can become part of a routine maintenance loop. That second conclusion leads directly to universal profiles and live module replacement in the following chapters.

Chapter 4

One Profile Fits All

This chapter follows the structure of the One Profile Fits All paper before drawing broader design lessons. The original paper first establishes that the Linux kernel is a front-end bottleneck for data-center workloads, then measures profile similarity at function and branch granularity, and finally evaluates a merged universal profile. The last section of the chapter interprets those findings as a deployment policy for kernel PGO rather than as a claim that specialization disappears.

4.1 Original Motivation: Kernel Front-End Bottlenecks

Modern data-center applications have large instruction footprints. Their user-space code already stresses instruction caches and instruction TLBs, and the Linux kernel adds a second large body of code that is frequently entered through system calls, networking, memory management, and scheduling. One Profile Fits All begins from this hardware observation. If a significant fraction of cycles and instruction-cache misses come from kernel instructions, then optimizing only user-space binaries leaves a major source of front-end stalls untouched.

The paper reports that the studied data-center applications spend 43–74% of their total CPU cycles in the kernel, 61% on average. It also reports that 43–79% of all I-cache misses and 10–86% of all I-TLB misses originate from the Linux kernel, with averages of 61% and 46% respectively [13]. These results justify the core claim in the presentation: the kernel is not a small background component for data-center performance.

These hardware-event results also explain why code layout is the main optimization target. If the kernel contributes a large share of instruction-cache and I-TLB misses, then the compiler can improve performance by shrinking the hot instruction footprint and placing frequently executed paths close together. This connects One Profile Fits All back to Tarax: both papers rely on PGO, but One Profile Fits All begins by proving that the kernel front end is a common bottleneck across several applications rather than an application-specific accident. The workload set and the hardware-event evidence are shown in Tables 4.1 and 4.2, following the benchmark design and measurements reported by Ugur et al. [13].

4.2 Original Methodology: Profile Collection and Similarity Metrics

One Profile Fits All uses similarity metrics to transform a qualitative question into a measurable one. The qualitative question is simple: do different data-center applications use the

Table 4.1: Applications and benchmarks used in One Profile Fits All.

Application	Version	Benchmark
Apache	2.4.41	ApacheBench
Nginx	1.18	ApacheBench
Redis	5.0.7	Redis benchmark
Memcached	1.5.22	Redis benchmark
LevelDB	1.22	db_bench
RocksDB	6.15.2	db_bench
MySQL	8.0.23	sysbench
PostgreSQL	12.5	sysbench

Table 4.2: Fraction of hardware events originating in the Linux kernel.

Application	Cycles	Instructions	I-cache misses	I-TLB misses
Apache	62.88%	64.55%	60.37%	63.11%
Nginx	74.02%	74.86%	68.30%	80.45%
Redis	69.84%	56.96%	78.88%	85.55%
Memcached	69.07%	55.04%	74.31%	51.62%
LevelDB	43.05%	30.27%	56.67%	31.70%
RocksDB	46.34%	41.79%	42.97%	18.19%
MySQL	63.26%	59.21%	64.86%	10.38%
PostgreSQL	60.00%	73.22%	43.85%	24.40%

kernel in the same way? The measurable version requires feature vectors. For function execution frequency, each feature represents how often a particular kernel function executes under a workload. For branch execution frequency, each feature represents how often a branch instruction executes. For conditional branch behavior, the features represent taken and not-taken frequencies.

Cosine similarity is appropriate for comparing frequency distributions because it measures whether two vectors point in the same direction. If two applications call the same kernel functions in similar proportions, their cosine similarity will be high even if the absolute request rate differs. This is useful in data centers because benchmark throughput can change across services and machines. What matters for code layout is not only how many requests are served, but which kernel paths dominate the instruction stream.

The paper also uses vector norms for conditional branch behavior. This complements cosine similarity because branch taken/not-taken data are bounded and can be compared as differences. A small L0 norm means few branch outcomes differ. A small L1 norm means the total magnitude of the differences is small. The presentation's use of confusion matrices makes the conclusion easy to see visually: many services share a strong common kernel control-flow shape.

The sequence of metrics is important. Function frequency similarity answers whether applications spend time in the same kernel functions. Branch frequency similarity answers whether they execute similar internal control-flow edges. Conditional branch similarity answers whether the compiler's likely and unlikely paths would be similar across applications. The paper therefore moves from coarse evidence to fine-grained evidence. This progression makes the universal-profile conclusion more convincing than a single aggregate speedup number would be.

Why kernel similarity is plausible. At first, it may seem surprising that Apache, Redis, RocksDB, MySQL, and PostgreSQL could have similar kernel profiles. Their user-space algorithms are different. However, from the kernel's perspective, many data-center services repeatedly exercise a common set of primitives. They accept connections, move data through sockets, allocate and free memory, wake and block threads, poll events, use timers, and interact with storage or page cache mechanisms. These operations are implemented by shared kernel subsystems.

The similarity claim does not mean that all services are identical. One Profile Fits All explicitly observes exceptions and lower-similarity cases. The important point is that the shared portion is large enough to make a universal profile useful. The kernel contains many paths that are cold for almost every data-center service. Moving those paths away from the hot layout and arranging the common hot paths well can benefit multiple applications at once.

This observation also helps explain why universal kernel PGO is different from universal user-space PGO. A universal profile for unrelated user-space applications would usually make little sense because their code bases are different. The Linux kernel, by contrast, is the same code base shared by all applications on the machine. The question is not whether the applications are similar in their own logic, but whether their entries into the shared kernel text produce similar hot code. One Profile Fits All shows that, for many data-center applications, they do.

The exceptions are also informative. Lower similarity for storage-oriented workloads does not contradict the universal-profile idea; it identifies where the boundary of the idea lies. The shared profile is strongest when services repeatedly exercise common network, event, scheduling, and memory paths. It is weaker when an application spends more time in a distinctive storage or database path. This is why a production system should treat a universal profile as a default profile class, not as a proof that all specialization is unnecessary.

4.3 Original Results and Universal Profile

Tarax suggests that each application deserves its own optimized kernel. One Profile Fits All asks whether that assumption is too strict. If different data-center applications use the kernel in similar ways, then a profile collected from multiple workloads may be effective for a broad class of services. The paper compares kernel profiles using feature vectors of function execution frequencies and cosine similarity. A high cosine similarity indicates that two applications spend kernel execution effort in similarly distributed functions. As shown in Figure 4.1, Ugur et al. [13] find high function-frequency similarity for most of the studied services.

The result is striking: with the exception of LevelDB and RocksDB, the applications show close similarity in kernel function usage. This does not mean the applications are identical at the user-space level. Apache and MySQL are obviously different systems. The point is that once their execution enters the Linux kernel, the hot control-flow distribution has a shared structure. That shared structure creates room for a universal profile.

Branch similarity and conditional behavior. Function frequency is not the only profile signal relevant to compiler optimization. Branch execution frequency and conditional branch taken/not-taken behavior affect basic-block ordering, fall-through paths, and code layout. One Profile Fits All therefore also studies branch-level similarity. The presentation highlights a second confusion matrix for branch instruction behavior, again showing substantial similarity across most applications. This result is shown in Figure 4.2, while Figure 4.3 summarizes the conditional branch taken/not-taken differences reported by Ugur et al. [13].

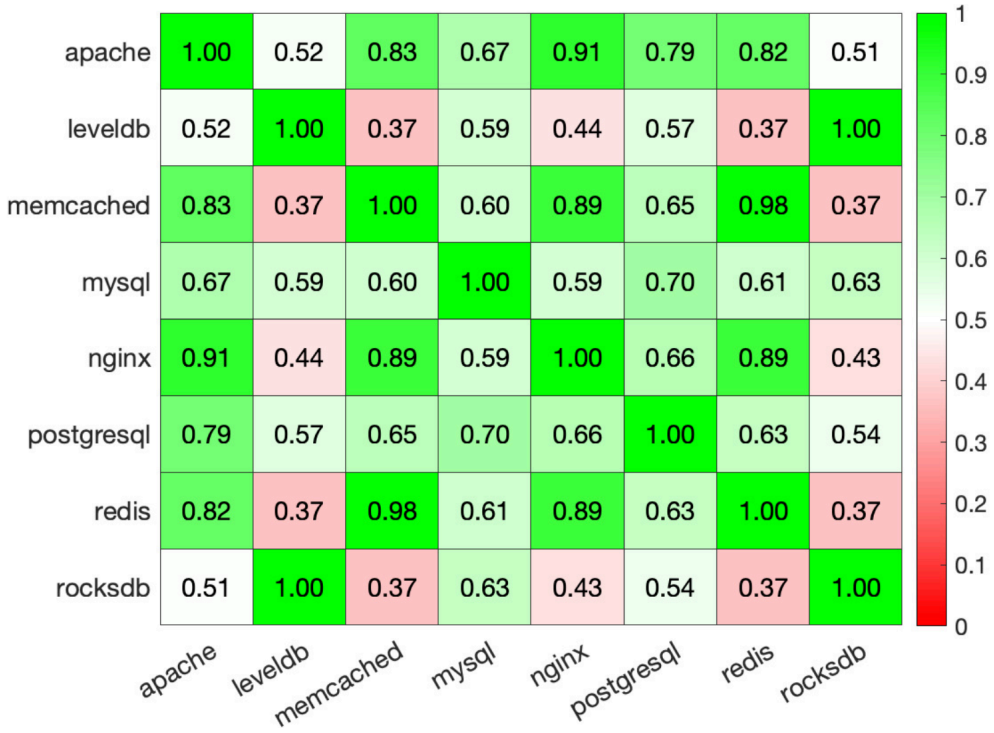


Figure 4.1: Cosine similarity matrix for kernel function execution frequencies.

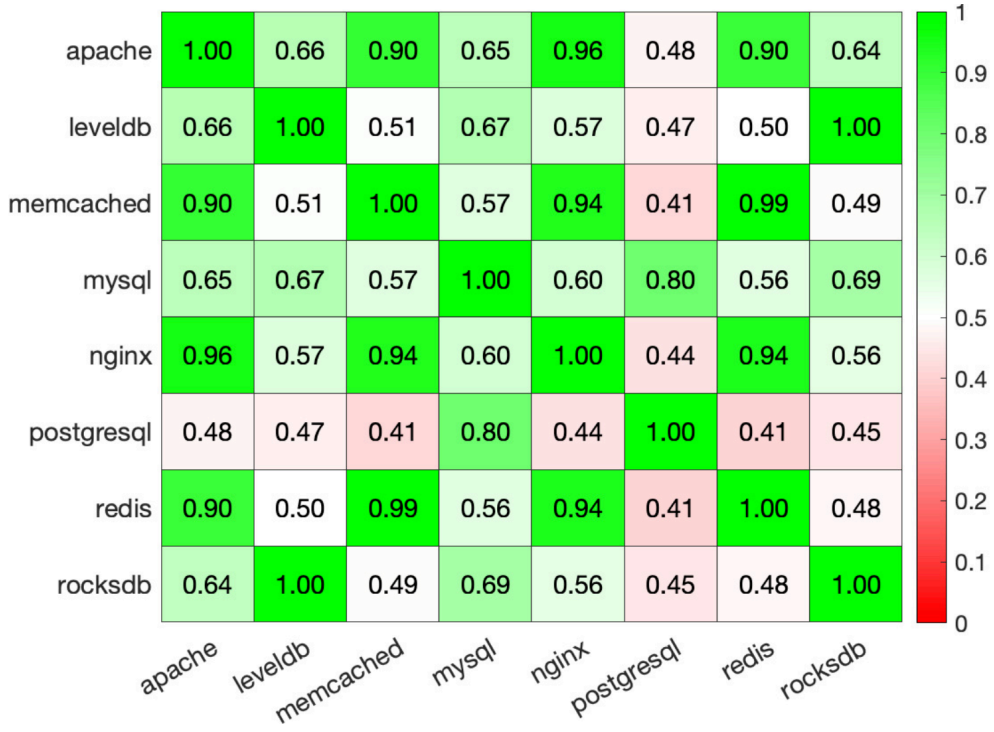


Figure 4.2: Cosine similarity matrix for kernel branch execution frequencies.

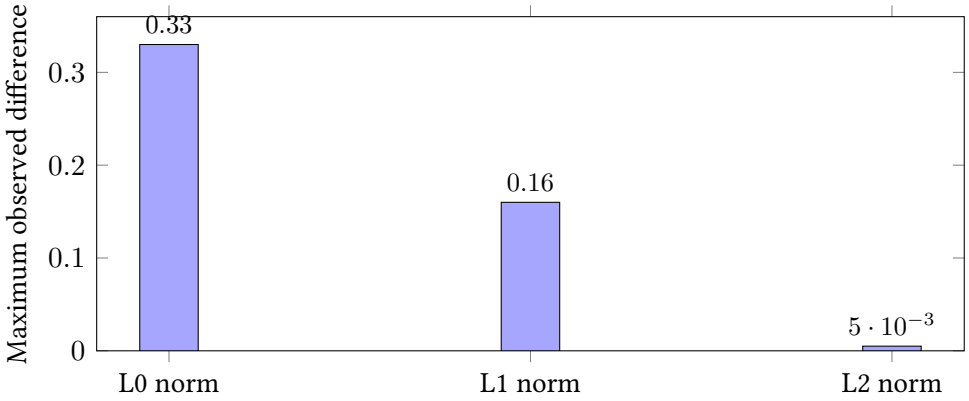


Figure 4.3: Summary of conditional branch taken/not-taken similarity.

The branch-level analysis matters because it strengthens the universal-profile argument. If similarity appeared only at the coarse function level, a merged profile might still mislead the compiler inside hot functions. Similarity at branch granularity suggests that code layout decisions driven by the universal profile are less likely to conflict with the behavior of individual applications.

This also explains why the paper evaluates both function execution and branch behavior before reporting speedup. The speedup is the final outcome, but the similarity analysis explains why the outcome is plausible. Without the similarity evidence, a universal profile could appear to be an empirical coincidence. With the similarity evidence, it becomes a design principle: shared kernel code plus shared service patterns can justify a shared optimization profile.

Universal profile construction. The universal profile is constructed by merging kernel execution profiles collected from the studied applications with equal weights. Equal weighting is a simple choice, and that simplicity is part of the appeal. A production operator does not need to maintain one kernel image for every service or estimate a delicate traffic-weighted mixture. Instead, the operator can build a kernel using a representative data-center profile bundle. The performance result in Figure 4.4 shows why this policy is plausible: the universal-profile kernel approaches the benefit of application-specific profiles in the evaluation of Ugur et al. [13].

The reported result is an average end-to-end speedup of 8.02% for the selected applications. Just as important, the universal profile achieves a speedup close to that of application-specific profiles. This is the main conceptual step beyond Tarax. The compiler still uses PGO, but the operational unit shifts from one profile per application to one profile for a class of data-center applications.

The operational implication is substantial. If a universal profile achieves most of the benefit of application-specific profiles, then operators can reduce the number of optimized kernel variants they maintain. The result does not remove the need for validation, but it changes the default policy. Instead of asking every service to justify a separate kernel, the operator can begin with a general data-center optimized kernel and reserve specialization for services that regress or fail to improve.

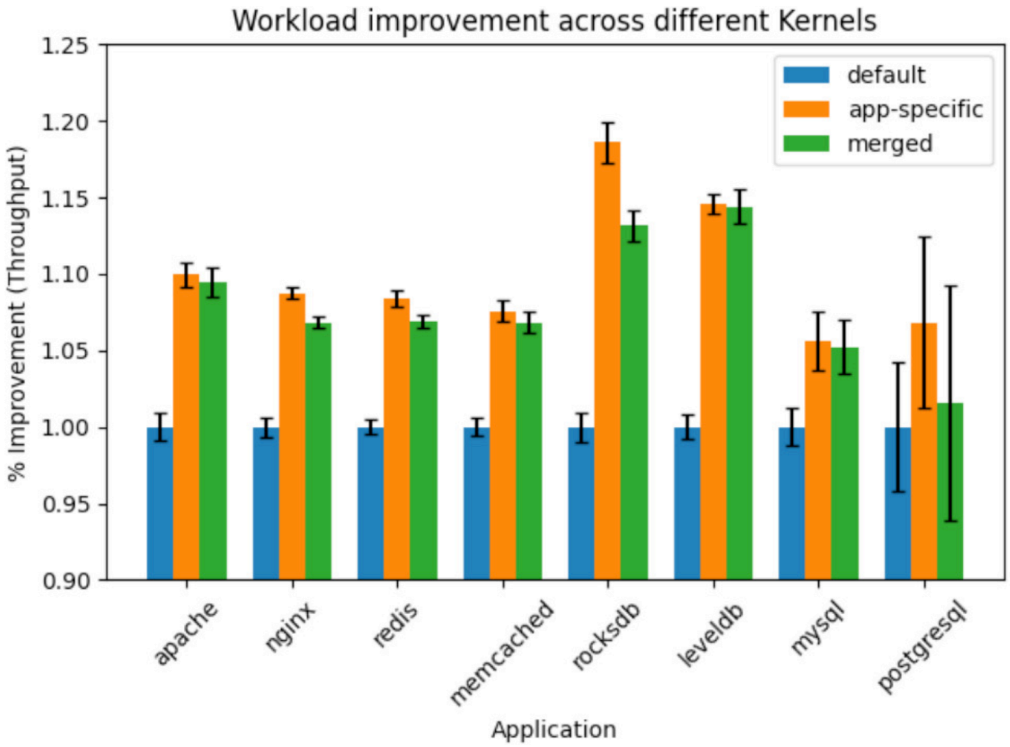


Figure 4.4: Performance comparison among different optimized kernels, including the universal-profile kernel.

4.4 My Analysis: Interpreting and Deploying Universal Profiles

The previous sections summarize the paper's original evidence: kernel execution is large enough to matter, kernel profiles are similar across many studied services, and a simple merged profile can recover much of the benefit of application-specific profiles. My analysis begins from a different question: how should a data-center operator use that evidence without overgeneralizing it?

The universal-profile result should be interpreted as an engineering trade-off rather than as a claim that application-specific profiles are unnecessary. A service-specific profile can still win when a workload has unusual kernel behavior. For example, a database with a particular storage pattern or an in-memory cache with an unusual request distribution may stress paths that are underweighted in the universal profile. The universal profile is valuable because it reduces the operational cost of kernel PGO while preserving most of the measured benefit for the studied workloads.

Equal weighting is also a conservative design choice. It avoids letting one high-throughput benchmark dominate the merged profile. In production, an operator could imagine other weighting policies: weight by fleet size, by CPU consumption, by revenue-critical services, or by tail-latency sensitivity. The paper's equal-weight result is therefore a lower complexity baseline. It shows that even a simple merging strategy can produce useful speedup. More adaptive strategies might improve performance, but they also introduce policy complexity.

The result is especially important for cloud providers. A provider may not want to expose a different kernel image for every tenant or every internal service. A universal data-center kernel profile offers a middle ground: the kernel is still optimized for data-center behavior rather than for generic desktop behavior, but it does not require per-application deployment.

In this thesis, that middle ground is the bridge between Tarax's application-specific kernel and PlugPGO's deployable module.

Limits of universal profiles. Profile generalization has limits. First, similarity is empirical. It depends on the selected applications, versions, benchmarks, kernel version, compiler, and hardware platform. Second, a profile can become stale when the workload changes. A merged profile built from one generation of data-center software may not represent a later generation with different networking stacks, storage engines, or accelerator usage. Third, a universal profile may hide minority workloads whose performance matters even if their fleet share is small.

These limits suggest that universal profiles should be part of a feedback loop. A provider could maintain a default universal profile and monitor applications that regress under it. Those applications could receive a specialized module or a different profile class. This is another reason PlugPGO is useful: if optimized components can be deployed live, the system can afford to experiment with profiles more frequently. Static whole-kernel deployment makes profile experimentation expensive; module-level deployment makes it operationally plausible.

Why universal profiles do not eliminate specialization. The universal-profile result is easy to misunderstand. It does not imply that all data-center workloads should always use the same optimized kernel. Instead, it shows that a common optimized baseline is possible. This distinction is important for system design. A universal profile is best viewed as the default profile for a class of workloads, not as the final profile for every workload forever. It captures the shared hot kernel behavior among common services, but it cannot anticipate every traffic pattern, configuration, storage device, or kernel option.

Specialization remains useful in at least three cases. The first case is a workload with a distinctive kernel path. A storage engine that uses direct I/O differently from the benchmarks, a network service that relies on a specialized packet-processing path, or a database configured for unusual synchronization behavior may spend time in kernel code that the universal profile considers cold. For such a workload, the universal profile might still improve common paths, but it could under-optimize the most important service specific paths.

The second case is a workload with a strict latency objective. A small throughput improvement is not always the right metric. If a profile moves code in a way that improves average throughput but worsens a rare tail-latency path, the result may be unacceptable for a latency-critical service. Application-specific profiling gives operators more control over such trade-offs. A universal profile can be the safe default, but a service with important tail-latency constraints may still require its own profile or at least its own validation group.

The third case is hardware specialization. The One Profile Fits All experiments are tied to a particular hardware and software setting. In real data centers, kernel behavior changes across CPU generations, memory hierarchies, NICs, storage devices, and virtualization layers. An optimized layout that works well on one platform may not be ideal on another. Universal profiles can therefore be universal within a profile class rather than universal across the entire fleet. A provider might maintain one profile for general web-serving machines, another for storage-heavy machines, and another for virtualized hosts.

This interpretation makes the connection to PlugPGO stronger. If optimized modules can be updated online, operators do not need to choose permanently between universal and application-specific profiles. They can deploy a universal module as the default, observe service behavior, and then replace it with a specialized module for workloads that need it. The live-update mechanism lowers the cost of moving along the specialization axis. Without live update, every profile choice becomes a kernel-image decision. With live update, profile choice becomes a deployable module decision.

Strengths and limitations. One Profile Fits All reduces profile-management cost. Instead of asking operators to collect, validate, rebuild, and deploy a different optimized kernel for each application, it suggests that a shared data-center profile can be practically effective. This is especially valuable for cloud platforms that operate many services but want to maintain a small number of kernel variants.

The limitation is that profile generalization does not solve live deployment. A universal profile still produces a statically optimized kernel artifact. If deploying that artifact requires rebooting machines, then the method remains constrained by maintenance windows, rolling restarts, and availability risk. In other words, One Profile Fits All addresses the profile dimension of kernel PGO, but not the update dimension.

The practical value of the chapter is that it separates profile management from code deployment. Tarax makes profile management expensive because every application can lead to a separate kernel image. One Profile Fits All reduces that cost by showing that a shared profile can capture a large amount of common kernel behavior. However, the optimized output is still a kernel artifact. This means the universal profile should be seen as one layer in a larger system: it simplifies what profile to build, but it does not decide how the optimized code should be rolled out.

For a data-center operator, this suggests a realistic policy. A universal profile can be the default optimization for common services, while exceptional services can be detected through monitoring and assigned specialized profiles only when necessary. Such a policy would avoid both extremes. It would not force every application into its own kernel, and it would not assume that one profile is always sufficient. This middle-ground policy is especially compatible with PlugPGO because module-level update makes it easier to move between universal and specialized optimized code after deployment.

Chapter 5

PlugSched and Kernel Live Update

This chapter treats PlugSched first as an original live-update system and only then as a building block for hot-upgradable PGO. The first three sections follow the paper's own argument: existing live-update mechanisms are either too narrow or too disruptive, PlugSched creates a scheduler module boundary, and the evaluation demonstrates millisecond-level visible downtime. The final section then extracts the lessons that matter for PlugPGO.

5.1 Original Deployment Problem

The previous chapters establish that kernel PGO can improve performance and that a universal profile can reduce per-application tuning cost. However, data-center operators also care about how optimized code reaches production. A whole-kernel rebuild followed by reboot is an expensive deployment unit. It interrupts local state, requires orchestration, and raises the cost of frequent optimization. If kernel PGO is to become a closed-loop system, the optimized code must be deployable without treating every profile update as a kernel replacement event.

PlugSched is relevant because it targets one of the most tightly coupled Linux subsystems: the scheduler. If the scheduler can be modularized and replaced online, then component-level kernel update is more general than small function patches. PlugSched is not itself a PGO system; its original contribution is a deployment mechanism with a clear vocabulary: modularization, boundary analysis, function redirection, stack inspection, data rebuild, and rollback.

The connection to the previous chapter is direct. One Profile Fits All reduces the number of profiles that may need to be deployed, but it still assumes an optimized kernel image. PlugSched changes the deployment unit. It shows that even a subsystem with many kernel dependencies can be separated enough to support online replacement. This chapter therefore shifts the thesis from profile selection to update mechanics.

Limitations of existing live-update techniques. Existing live-update mechanisms often operate at function granularity. Tools such as kpatch and Ksplice can redirect individual functions, which is useful for security patches or small bug fixes. They are less expressive for replacing a whole subsystem whose behavior and internal state must change together. Component-level alternatives can support broader changes, but may require virtualization assumptions, kernel modifications, or additional runtime overhead. The presentation uses this comparison to motivate PlugSched's middle ground: component-level scheduler update without imposing extra context-switch overhead during normal execution. As shown in Figure 5.1 and Table 5.1, Ma et al. [6] position PlugSched between narrow function patching and more disruptive component-level alternatives.

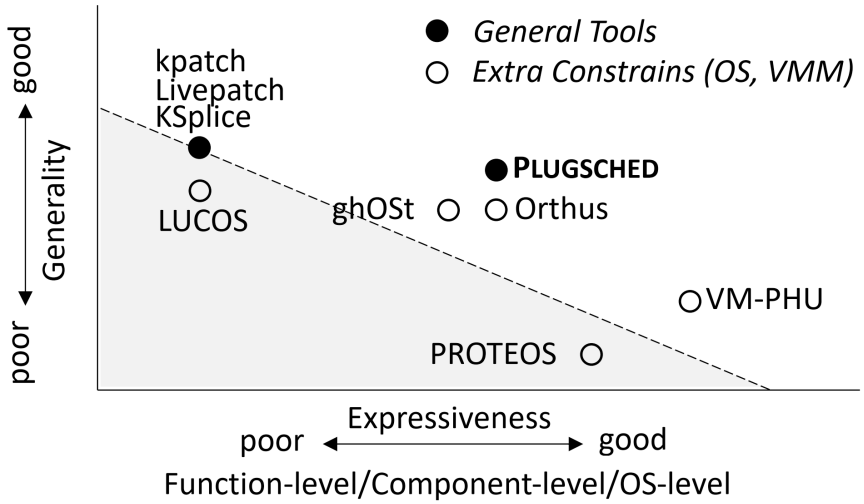


Figure 5.1: Comparison between potential live-update solutions.

Table 5.1: Potential solutions for scheduler update.

Approach	Expressive	No kernel changes	No extra overhead	Granularity
eBPF	No	Yes	Yes	Function
kpatch	No	Yes	Yes	Function
ghOSt	Yes	No	No	Component
Kernel module	Yes	Yes	Yes	Component
PlugSched	Yes	Yes	Yes	Component

The key constraint is safety. Replacing scheduler code while tasks are executing can corrupt stacks, leave data structures inconsistent, or redirect execution into code that has already been freed. PlugSched therefore treats update as a coordinated process rather than a simple jump replacement. It must know which functions belong to the scheduler, which data are internal or external, and when it is safe to switch from old code to new code.

This comparison also clarifies the system gap that PlugSched addresses. Function patching is useful when the change is local, but a scheduler replacement can span many helper functions and data relationships. A module-based mechanism gives the update system a larger region to replace while still avoiding whole-kernel reboot. PlugSched's position between function patching and whole-kernel replacement is therefore the key design point of the paper.

5.2 Original PlugSched Design: Requirements and Modularization

PlugSched's design is driven by three requirements. The first requirement is expressiveness. A scheduler update may be a small patch, a feature change, or a replacement with a different scheduler. Function-level live patching can handle small changes, but it is not expressive enough when the update spans multiple files, internal helper functions, and scheduler-private data structures.

The second requirement is compatibility with existing production kernels. A live-update system that requires deep kernel modifications may be difficult to deploy in commercial Linux distributions. PlugSched therefore tries to modularize the scheduler without changing the

normal running cost of scheduling. This is a key contrast with approaches that move scheduling policy to user space. Programmability is useful, but a data-center operator may not accept a permanent context-switch overhead on every scheduling decision.

The third requirement is short downtime. The critical stop-the-machine interval must be small enough that latency-sensitive workloads do not experience visible service degradation. The paper's reported millisecond-level downtime is therefore not just an implementation statistic; it is the condition that makes component-level update credible. A live-update mechanism that stops service execution for too long would be rejected even if the replacement code is functionally correct.

These requirements are mutually dependent. Expressiveness without compatibility would produce an elegant but hard-to-deploy system. Compatibility without expressiveness would fall back to small patches that cannot replace a meaningful optimized component. Short downtime without rollback would still be risky because performance regressions can appear only after the update reaches real traffic. PlugSched is valuable because it treats these requirements as one design problem.

Scheduler modularization. The Linux scheduler is not naturally a standalone module. It interacts with memory management, timers, cgroups, CPU hotplug, architecture-specific code, and many hot paths. PlugSched modularizes it by classifying functions into interface, internal, and external categories. It gathers scheduler-relevant information during compilation, analyzes the call graph to determine boundaries, and extracts scheduler code into an independent module directory. The workflow in Figure 5.2 shows the overall transformation, and Figure 5.3 shows how boundary analysis separates scheduler-internal code from external kernel dependencies [6].

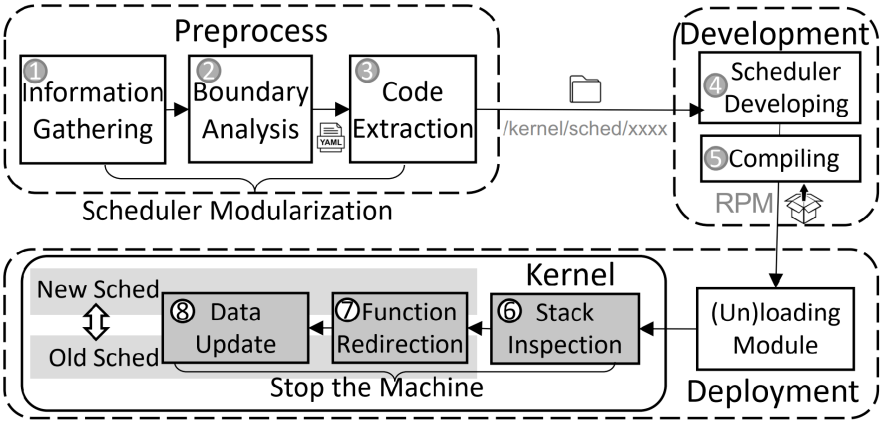


Figure 5.2: Workflow of PlugSched.

The modularization step is the conceptual counterpart to PGO's profiling step. Before a module can be optimized and replaced, the system must define what the module is. A poor boundary would either exclude important scheduler behavior or include too many kernel dependencies. PlugSched's boundary analysis is therefore not merely a build-system trick; it is the foundation for component-level update.

The paper's scheduler case is also a stress test for the idea of modular live update. The scheduler is involved in context switches, task wakeups, load balancing, and CPU management. If such a subsystem can be given a usable update boundary, then less tightly coupled kernel components become plausible targets for other component-level kernel updates. This makes the scheduler a strong stress test for the paper's mechanism.

Function redirection and state migration. Once the scheduler has been modularized, PlugSched redirects execution from old functions to new module functions. Redirection is

Algorithm 1: Boundary Analysis.

(1) Initialization:
Load user-defined configurations ($F_{interface}$) and compiler generated information (F_{mod}, G)
(2) Inflect:
$F' \leftarrow F_{mod} - F_{interface}$
$F'_{external} \leftarrow \emptyset$
while *True* **do** /* Coverage */
  $F^* \leftarrow \emptyset$
  foreach $e_{f_i,f_j} \in G$ **do**
    if $f_i \notin F' \wedge f_i \notin F_{interface} \wedge f_j \in F'$ **then**
      $F^* \leftarrow F^* \cup \{f_j\}$
    end
  end
  if $F^* = \emptyset$ **then**
    break
  end
  $F'_{external} \leftarrow F'_{external} \cup F^*, F' \leftarrow F' - F^*$
end
(3) Save Results:
$F_{internal} \leftarrow F', F_{external} \leftarrow F'_{external}$

Figure 5.3: Scheduler modularization and boundary analysis.

implemented by replacing function prologues with jump instructions. For most functions this is straightforward, but `__schedule()` is special because it participates in context switching and stack management. PlugSched handles this with stack-aware techniques so that execution can continue safely after the switch.

State migration is equally important. Some scheduler data are external and must be inherited from the existing kernel state. Some are internal and can be rebuilt or privately allocated by the new scheduler. The PlugSched paper classifies these data types and defines how each is handled. This is a major difference from simple function patching: component-level update must preserve both control flow and data semantics.

This state classification is one of the most important parts of the original paper. It shows that component-level live update is not only a control-flow problem. The replacement code must also agree with the meaning and ownership of kernel data. PlugSched handles the harder case in which a scheduler update may add, remove, or rebuild state, which is why the paper can support both patches and feature-level scheduler changes.

Handling the scheduler boundary. The scheduler boundary is difficult because the scheduler is a hot subsystem rather than a clean library. Many kernel components call scheduler functions, and scheduler functions call back into other subsystems. PlugSched uses information gathering and boundary analysis to divide functions into interface, internal, and external categories. Interface functions form the boundary through which the rest of the kernel interacts with the scheduler module. Internal functions can move into the module. External functions remain in the kernel and are called by the module.

This classification is directly relevant to the paper's update granularity. If the boundary is too small, the mechanism degenerates into ordinary function patching. If it is too large, state migration and dependency management become unsafe. PlugSched shows that a useful

middle boundary can be discovered for a complex subsystem.

The boundary also determines rollback. A live update is only practical if the system can return to the previous module after a failed deployment. This means the old entry points, old code, and compatible data state must remain available until the new module is known to be safe. A whole-kernel PGO deployment often rolls back by booting a previous kernel. A module-level deployment can roll back by redirecting calls back to the old module, which is much faster if the boundary is well-defined.

Stack inspection and stop-the-machine scope. PlugSched uses stack inspection to check whether threads are currently executing code that is about to be replaced. A naive stack inspection could be expensive because it would need to examine many active threads and deep call chains. The paper therefore optimizes the process with parallelism and search techniques. The goal is not to eliminate the check, but to shrink the time during which the machine must be stopped.

This design matters because the visible downtime is not the same as the total update time. Initialization, module preparation, and cleanup can happen while applications keep running. The critical downtime is the short interval when redirection and consistency checks require stronger synchronization. PlugSched's evaluation shows that moving uncritical work out of this interval is essential: preparation can be long, but visible downtime must be short.

The stack-inspection optimization is also a reminder that live update is a concurrency problem. It is not enough to install a jump to new code. The system must reason about what other CPUs are executing at the same time and whether any stack frame still depends on old code. In a data-center machine with many cores, this concurrent state is the normal case. This is why PlugSched keeps old code and state valid until the update can prove that execution has moved safely. The range of changes used to test these requirements is listed in Table 5.2, where Ma et al. [6] include optimization patches, bug patches, feature changes, and a new scheduler.

Table 5.2: Different cases used to evaluate PlugSched.

ID	Type	Commit ID/Feature	Files	LOC	Data
1	p(opt)	aa93cd53bc1b91	1	24	No
2	p(opt)	11f10e5420f6ce	1	3	No
3	p(opt)	45da7a2b0af8fa	1	4	No
4	p(bug)	b9c88f75226838	1	9	No
5	feature	Remove Bandwidth Control	1	1258	Yes
6	feature	Add Group Identity	7	2592	Yes
7	new scheduler	Tiny Scheduler	10	3845	Yes

5.3 Original Evaluation: Downtime and Service Impact

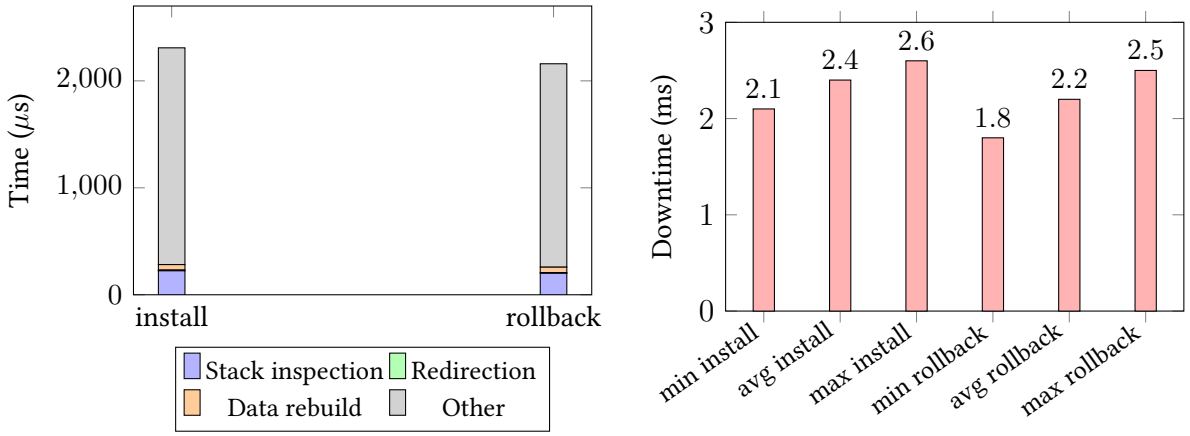
PlugSched reports millisecond-level downtime. For the seven update cases, downtime is 2.1–2.6 ms for installation and 1.8–2.5 ms for rollback, with averages of 2.4 ms and 2.2 ms respectively [6]. Table 5.3 shows the breakdown for the Tiny Scheduler case. The total update time is much larger than the downtime, but applications are only stopped during a small critical window.

The same update costs are visualized in Figure 5.4. The left subfigure separates stack inspection, redirection, data rebuild, and other downtime, while the right subfigure shows the

Table 5.3: Breakdown of total time in PlugSched for the Tiny Scheduler case.

Phase	Upgrade	Rollback
Total time	141 ms	93 ms
Initialization	137 ms	85 ms
Down time	2309 μs	2160 μs
Stack inspection without parallel optimization	1585 μs	1054 μs
Stack inspection with parallel optimization	224 μs	201 μs
Function redirection	7 μs	6 μs
Data rebuild without parallel optimization	135 μs	112 μs
Data rebuild with parallel optimization	51 μs	51 μs
Other downtime	2027 μs	1902 μs

install and rollback range reported by Ma et al. [6].



(a) Downtime composition

(b) Reported downtime range

Figure 5.4: Downtime summary for PlugSched update and rollback.

The service-level effect is shown in Figure 5.5. This figure matters because Ma et al. [6] evaluate not only the internal update cost but also the throughput observed by a running Nginx service during scheduler replacement.

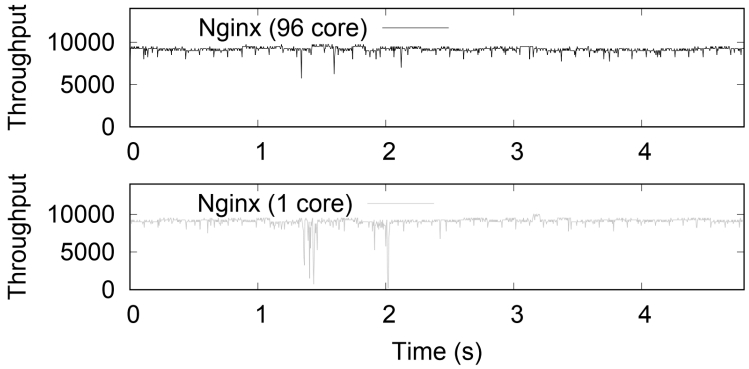


Figure 5.5: Nginx throughput during scheduler update.

The production behavior result is the most important part of the original evaluation. If live update caused large throughput collapse, scheduler modularization would be technically interesting but operationally weak. PlugSched's Nginx and Redis experiments instead suggest that the update window is short enough for service workloads to tolerate, especially compared with reboot-based deployment.

The downtime table and throughput figure should therefore be read together. The table shows where time is spent inside the update mechanism, while the throughput figure shows what the service experiences. This combination is what makes the evaluation persuasive: it measures both the internal cost of module replacement and the external cost seen by the application. A mechanism that looks efficient internally but causes a visible service disruption would not satisfy the data-center requirement.

5.4 My Analysis: Lessons for PlugPGO

The preceding sections describe PlugSched as the original paper presents it: a live-update system for replacing Linux scheduler components with short visible downtime. My analysis now uses that system as a deployment model for optimized kernel modules.

PlugSched's production lesson is that update mechanisms must be evaluated under service workloads, not only under microbenchmarks. The Nginx throughput experiment in the presentation is valuable because it shows what an application observes while the scheduler update occurs. For an operating-system optimization pipeline, user-visible behavior is the real acceptance criterion. A technically correct update that causes unacceptable tail latency would still fail operationally.

PlugPGO should therefore evaluate both the update path and the optimized steady state. The update path asks how long the server pauses, whether throughput dips, and whether rollback is fast. The optimized steady state asks whether the PGO module actually improves the workload after the update. These two evaluations can conflict. A module boundary that is easy to update may be too small to capture much PGO benefit. A boundary that captures more hot code may require more careful safety checks. This boundary trade-off is the core engineering problem for a hot-upgradable PGO kernel.

Another production lesson is that the system must coexist with existing patching tools. PlugSched includes conflict checking because another live-patch tool may have already redirected some of the same functions. PlugPGO would need the same discipline. If an optimized module and a security patch both modify a hot function, the platform must define precedence, reject the unsafe combination, or regenerate the optimized module from the patched source. This is why compiler optimization, security patching, and live deployment should be treated as one coordinated kernel-update pipeline.

Failure and rollback. A live-update mechanism must be designed around failure as carefully as around the successful update path. For PlugSched, the first class of failure is a boundary failure. If a function or data structure that should be internal to the scheduler is still used directly by the rest of the kernel, replacing the scheduler can leave unresolved dependencies or inconsistent assumptions. This is why boundary analysis is a core part of the design. For PlugPGO, the analogous failure occurs when an optimized module is treated as self-contained even though the compiler has changed assumptions about calls, data layout, or helper functions beyond the intended boundary.

The second class of failure is an in-flight execution failure. On a multicore machine, one CPU can be executing old scheduler code while another CPU installs the new redirection. If the old text is removed too early, the running CPU may return to invalid code. If the new code expects state that has not been rebuilt, it may observe an impossible state. The PlugSched design handles this with a carefully controlled update sequence. PlugPGO's delayed memory release inherits the same principle: the old version must remain valid until the system can prove that no execution path still needs it.

The third class of failure is a semantic failure. The update may be mechanically safe but behaviorally wrong. A new scheduler might harm fairness or latency. A PGO-optimized module might reduce throughput under a workload that was not represented in the profile. This failure mode cannot be eliminated by static boundary analysis alone. It requires monitoring and rollback. The system should treat every optimized module as a candidate improvement that must earn its place in production, not as an unquestioned upgrade.

Rollback semantics are therefore part of the interface. A robust live-update system needs to keep enough information to return to the previous version quickly. For PlugSched, this means the old scheduler code and relevant data must remain available until the new scheduler is accepted. For PlugPGO, rollback is simpler when the update changes only code layout and does not change persistent data. The system can redirect calls back to the previous module and continue using the same data structures. This is one reason the presentation emphasizes removing data restoration from PlugPGO's initial design.

The rollback model also changes how operators can use optimization. If rollback is slow or requires reboot, operators will be conservative and update rarely. If rollback is fast and local, operators can use canary deployment, compare performance across machines, and iterate on profiles. In other words, rollback is not merely a safety feature. It is what makes closed-loop kernel optimization operationally realistic.

Implications for kernel PGO. PlugSched contributes a deployment primitive, not a compiler optimization. Its relevance to kernel PGO lies in the idea that optimized code can be packaged as a replaceable kernel component. A profile-guided scheduler module, memory-management module, or I/O path would still require careful boundary definition and state handling, but the deployment unit would no longer have to be a whole kernel image.

This is the conceptual opening for PlugPGO. If PGO can identify hot kernel code and produce optimized module code, and if a PlugSched-style mechanism can redirect execution into that module, then data centers can move toward online kernel specialization. The next chapter develops this direction using the method proposed in the presentation.

The chapter also shows why a live-update system must be measured in two time scales. The first time scale is total update time, which includes initialization, loading, analysis, and preparation. This time can be tens or hundreds of milliseconds as long as the service continues to run. The second time scale is visible downtime, the short interval during which execution must be stopped or redirected. PlugSched's contribution is to make the visible interval small enough that applications such as Nginx and Redis can tolerate the update. For PlugPGO, the same distinction matters: compiling and validating a PGO module may be expensive, but those steps should happen before the online critical path.

This perspective also explains why the PlugSched figures belong near the deployment discussion rather than in the background chapter. The workflow and boundary diagrams show what must be prepared before the update, while the downtime and throughput figures show what the application experiences during the update. Reading them together turns live update from a low-level patching trick into an operating-system deployment model. PlugPGO inherits that model and narrows it to the case where the new code is optimized for the same semantics.

Chapter 6

PlugPGO: Hot-Upgradable PGO Modules

6.1 Problem Statement: From Static Kernel PGO to Hot Update

The presentation's "My Method" section identifies the main deployment limitation of current PGO kernels: they rely on whole-kernel replacement. A server collects profile information, recompiles the kernel, and then must boot into the optimized image. This workflow is acceptable for offline evaluation but conflicts with the operating model of large-scale services. Production systems prefer incremental rollout, fast rollback, and minimal downtime. A kernel PGO system that cannot update online is therefore difficult to place in an automated data-center pipeline.

PlugPGO is motivated by combining the strengths of the previous systems. From Tarax, it borrows the claim that profile-guided compiler optimization can improve Linux kernel performance for server applications [14]. From One Profile Fits All, it borrows the idea that profiles can be shared or merged across related data-center workloads [13]. From PlugSched, it borrows the deployment mechanism of modularization and live replacement [6]. The result is a hot-upgradable module acceleration direction for Linux kernel PGO, as shown in Figure 6.1.

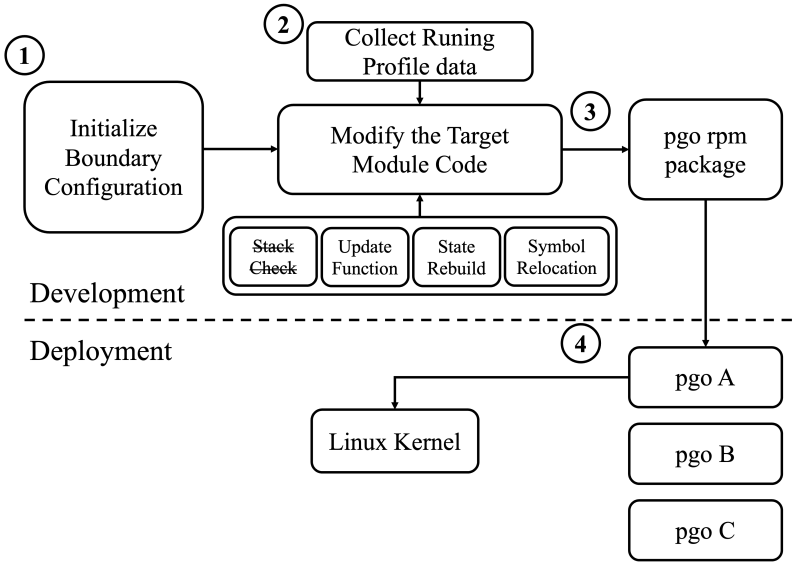


Figure 6.1: Workflow of PlugPGO.

6.2 PlugPGO Architecture and Online Optimization Loop

PlugPGO can be understood as a two-loop system. The outer loop collects runtime profiles from data-center workloads and recompiles selected kernel code with PGO. The inner loop deploys the optimized code as a replaceable module. Instead of waiting for a maintenance window to reboot the entire kernel, the system redirects calls from the old module to the optimized one and keeps enough old code alive until it is no longer executing. Figure 6.2 reformulates that workflow as a closed online loop: profile collection leads to module compilation, validation, redirection, delayed release, and runtime observation.

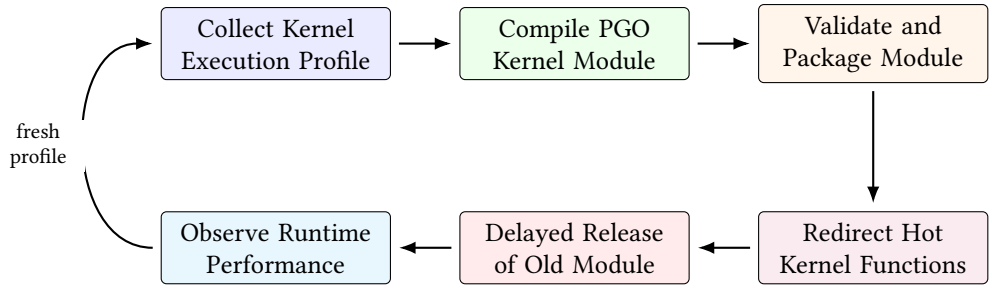


Figure 6.2: Static PGO becomes an online optimization loop when combined with modular live replacement.

This design intentionally avoids introducing new persistent kernel state when possible. If the optimized module reuses the system's current data structures and does not create new variables that require semantic migration, then the update path can be simpler than a full PlugSched scheduler replacement. The presentation therefore removes data restoration and active stack inspection from the proposed fast path. This does not mean that execution safety disappears; it means the first PlugPGO design relies on conservative target selection and delayed release rather than on rebuilding scheduler-like state during the online window. The trade-off is that the method is easiest to apply to PGO-driven code-layout and function-level optimization where data shape remains compatible.

6.3 Module Targeting, Compilation, and Safety

The first design decision in PlugPGO is selecting the module boundary. A good target should satisfy three conditions. First, it should be hot enough that compiler optimization can affect end-to-end performance. Second, it should have a stable interface to the rest of the kernel. Third, it should avoid complex state migration. These conditions follow directly from the three prior systems. Tarax shows that hot kernel paths matter. One Profile Fits All shows that common hot paths can exist across workloads. PlugSched shows that live update is feasible when boundaries and data ownership are explicit.

Not every kernel subsystem is an equally good target. A function that executes rarely is not worth optimizing dynamically. A subsystem with many global data-structure changes may require PlugSched-style data rebuild. A path that relies heavily on architecture-specific assembly may be difficult to compile and redirect safely. The initial PlugPGO design should therefore focus on stable code-heavy paths where PGO changes code layout and branch behavior but does not change the meaning of data.

This target-selection rule also explains why UnixBench pipe and swap are useful preliminary experiments. They are simple enough to isolate whether the dynamic replacement mechanism changes performance. They do not prove the full data-center case, but they help

identify the overhead of replacing optimized code dynamically. Once the mechanism is validated, the next step would be to evaluate larger services such as Nginx, Redis, or MySQL, matching the workload set used by Tarax and One Profile Fits All.

Profile collection and module compilation. PlugPGO needs a profile collection stage similar to Tarax, but the profile target is a module rather than an entire kernel image. This distinction changes the compiler problem. Whole-kernel PGO can optimize global function layout across the kernel. Module PGO is more constrained, but it may still improve local layout, inlining decisions, branch direction, and hot basic-block placement inside the module. The benefit depends on how much of the workload's kernel time is captured by the module boundary.

The profile can be application-specific or universal. An application-specific PlugPGO module would be built from a profile collected under one workload, similar to Tarax. A universal PlugPGO module would be built from a merged profile, similar to One Profile Fits All. A hybrid system could maintain both: a default universal optimized module for most servers and specialized modules for workloads that are known to deviate. The live-update mechanism makes this hybrid design more realistic because modules can be rolled out and rolled back without rebooting.

The compilation stage must preserve compatibility. The module's exported symbols, calling conventions, and expected data layouts must match the running kernel. This is why PlugPGO is not equivalent to compiling a new kernel object with arbitrary flags. The compiler must be constrained in the same spirit as Tarax constrains GCC for kernel PGO. The output should be faster code for the same interface, not a semantically different kernel component.

Removing data restoration and active stack inspection. PlugSched needs data rebuild because a new scheduler can add features, change structures, or maintain different internal queues. PlugPGO has a narrower purpose: it deploys an optimized version of code that is intended to preserve the same data semantics. If all data use the system's current structures, then no state migration is required. This is the reason the presentation removes the data restoration phase. In effect, PlugPGO treats PGO as a code optimization rather than a data model update.

The presentation also removes active stack inspection from the online path. This assumption is plausible only when the replacement target is selected so that no stack frame needs to be transformed and no execution path jumps into a function with incompatible frame layout. The simplification should therefore be treated as a design constraint rather than as a general live-update rule. PlugPGO should restrict itself to modules and functions where stack compatibility can be guaranteed by construction, compiler options, conservative update timing, and delayed release of old code. If future versions update more invasive kernel code, a PlugSched-like stack inspection phase may need to be restored.

Delayed memory release. Delayed memory release is the core safety mechanism retained by PlugPGO. After redirection, new calls enter the optimized module, but some CPU may still be executing instructions from the old module. Freeing the old code immediately could cause a crash or undefined behavior. PlugPGO therefore keeps the abandoned code resident until it can be safely reclaimed. This is analogous to read-copy-update style thinking: publish the new version, allow old readers or executions to drain, and reclaim old memory only after the grace period.

Delayed release also highlights the difference between update latency and memory overhead. Keeping old modules alive temporarily consumes memory, but the cost is small relative to rebooting a server or blocking a service. The design is therefore aligned with data-center priorities: prefer a small, bounded temporary resource cost over downtime.

6.4 Deployment, Evaluation, and Trade-Offs

The PlugPGO deployment algorithm can be described as a conservative sequence of checks and transitions. First, the system selects a target module and collects a profile under either an application-specific workload or a universal workload bundle. Second, it compiles an optimized module from the same kernel source and configuration as the running kernel. Third, it validates that the new module preserves the required external interface. Only after these offline steps does the online replacement begin. This ordering keeps the short online window focused on redirection rather than compilation or profile processing.

During the online phase, PlugPGO installs the optimized module into memory but does not immediately publish it to the rest of the kernel. It then prepares function redirection entries, verifies that the old and new symbols match, and switches entry points so that new invocations reach the optimized version. Because the old version may still be active on another CPU, the system records it as retired rather than freeing it. The retired module is released only after a grace period or another conservative quiescence condition. If the new module misbehaves, the redirection can be reversed and the retired module becomes active again. This deployment sequence is summarized in Table 6.1, which makes the safety condition explicit for each phase.

Table 6.1: Conceptual PlugPGO deployment sequence.

Phase	Main action	Safety condition
Profile	Collect execution profile	Profile matches target kernel and workload class
Build	Compile optimized module	ABI, symbols, and data layout remain compatible
Load	Place new module in memory	New text is present but not yet published
Redirect	Switch calls to new module	Entry points are atomically updated
Observe	Monitor service behavior	Throughput, latency, and error metrics stay acceptable
Retire	Keep old module temporarily	Old executions can finish safely
Commit or rollback	Free old module or restore it	Decision follows validation and monitoring

This algorithm is intentionally stricter than a simple module reload. The goal is not to change kernel functionality, but to change the compiled form of already existing functionality. That narrower purpose lets PlugPGO avoid some of PlugSched's expensive state handling, but it also imposes discipline. The optimized module should be generated from a source version that matches the running kernel. Compiler flags should be recorded so that the operator can reproduce the artifact. The profile should be tagged with the workload, kernel version, hardware class, and time of collection. Without this metadata, operators cannot know whether a module is a safe optimization for a given server.

The deployment algorithm also provides a natural place for staged rollout. A data center does not need to install a new PGO module everywhere at once. It can first deploy the module to a canary group, compare it against unmodified machines, and then expand the rollout if the result is positive. Because rollback is fast, the canary group can be small and the experiment can be frequent. This is a major advantage over whole-kernel PGO, where even a canary deployment usually requires rebooting machines into a new kernel image.

Integration with universal profiles. PlugPGO can use both application-specific and uni-

versal profiles, and this flexibility is important for deployment. A universal profile is a good default for code that sits on a common hot path shared by many services. For example, networking, scheduling, and memory-management paths often appear across web servers, caches, and databases. If a module covers such a path, a universal profile may provide a useful baseline speedup with minimal profile-management burden. The same optimized module can be deployed across a large part of the fleet.

Application-specific profiles remain useful for services that deviate from the baseline. PlugPGO can support this without requiring a separate whole-kernel image. A storage service, for example, might use a specialized module optimized for its I/O path, while a web-serving fleet uses the universal module. The important difference from Tarax is that the specialization unit is smaller. The data center can maintain a small number of optimized modules for different profile classes rather than a large number of complete kernel images.

This design suggests a practical profile hierarchy. At the top is the stock kernel, which serves as the compatibility and rollback baseline. The next layer is a universal data-center PGO module, built from a representative mixture of services. The final layer contains specialized modules for workloads that justify their own profile. Monitoring decides when a server should stay on the universal module and when it should move to a specialized one. Because the update is online, the system can make that decision gradually rather than at boot time.

The hierarchy also gives a clean answer to profile staleness. A stale specialized profile does not have to endanger the whole kernel. If performance monitoring detects that the specialized module no longer helps, the system can fall back to the universal module or the original module. This makes profile freshness a manageable operational problem rather than a reason to avoid kernel PGO altogether.

Preliminary results. The presentation evaluates PlugPGO with UnixBench microbenchmarks, focusing on pipe and swap. These benchmarks are intentionally simple. They do not prove that PlugPGO will preserve all benefits of static PGO for large applications, but they are useful for testing the mechanism's first-order effect: dynamic replacement can execute optimized kernel code, yet it may lose part of the speedup achieved by a statically optimized kernel. As shown in Tables 6.2 and 6.3, the presentation uses swap and pipe results to compare static PGO, dynamic replacement, and the baseline kernel. Table 6.4 then summarizes the design trade-off implied by those preliminary measurements.

	Time	Branch-miss	Cache-miss
Org	137.75 us	0.346%	1.05%
Static Replacement	129.84 us	0.334%	0.98%
Dynamic Replacement (Ours)	133.56 us	0.341%	1.02%
Improvement	↓ 4.19 us	↓ 0.005%	↓ 0.03%

Table 6.2: Swap performance comparison.

	Time	Branch-miss	Cache-miss
Org	120.34 us	0.372%	1.01%
Static Replacement	115.46 us	0.365%	0.98%
Dynamic Replacement (Ours)	119.94 us	0.373%	1.00%
Improvement	↓0.4 us	↑0.001%	↓0.001%

Table 6.3: Pipe performance comparison.

Table 6.4: Static PGO and PlugPGO trade-off.

Dimension	Static kernel PGO	PlugPGO-style dynamic module PGO
Deployment unit	Whole kernel image	Selected kernel module or component
Update method	Reboot or planned replacement	Live function/module redirection
Expected speedup	Highest, because layout is global	Lower, due to indirection and boundary limits
Availability	Requires maintenance orchestration	Millisecond-level target downtime
Rollback	Boot previous kernel or redeploy image	Redirect back to previous module
Best use case	Stable long-lived kernel variant	Frequent optimization and canary rollout

Safety invariants for hot replacement. A PlugPGO update should satisfy several safety invariants. The first invariant is interface equivalence: every function that remains visible to the rest of the kernel must preserve its calling convention and expected side effects. PGO may change layout, inlining, and branch order, but it must not change the semantic contract of the module. This invariant is what allows the old and new modules to be swapped through redirection.

The second invariant is data compatibility. If the optimized module reads or writes kernel data structures, their layout and ownership must be the same as in the old module. The presentation's removal of data restoration relies on this invariant. If a future version of PlugPGO allows optimized modules to change internal data layout, then a PlugSched-style data rebuild phase would become necessary. For the first version, avoiding such changes keeps the update mechanism simpler and safer.

The third invariant is execution quiescence for old code. Delayed memory release ensures that old code is not freed while it might still be executing. This invariant is especially important on multicore machines, where one CPU may enter the old function just before another CPU installs the redirection. The update mechanism must account for such races. Keeping the old module alive until all possible old executions have drained is a conservative way to avoid use-after-free failures.

The fourth invariant is rollback availability. A PGO module is an optimization, not a correctness requirement. If monitoring detects a regression, the system should be able to redirect execution back to the previous version. This is one advantage of treating PGO as a live module rather than as a whole-kernel image. Rollback can be fast and local, which encourages more frequent experimentation with profiles.

Performance interpretation. The presentation states that dynamic replacement may lose part of the performance advantage of static PGO. This is expected. Static PGO can optimize across a larger code region and may benefit from global layout decisions. A dynamic module has a boundary, and redirection may add a small cost. Moreover, the compiler may have fewer opportunities to inline across the module boundary. Therefore, PlugPGO should not be judged only by whether it equals the best static PGO kernel.

The correct comparison is operational. Suppose static PGO gives a larger speedup but requires a reboot, while PlugPGO gives a smaller speedup but can be deployed online. For a production cluster, the second option may be preferable because it can be rolled out more often and with lower risk. A 5% improvement that can be safely applied to thousands of

servers may be more valuable than an 8% improvement that waits for a maintenance window. This is the same reasoning that makes AutoFDO valuable: practical deployment can matter as much as peak optimization quality.

The preliminary UnixBench results should therefore be read as a mechanism check. They show whether the optimized dynamic path behaves as expected, not whether PlugPGO has completed the full data-center story. The natural next experiment would combine the Tarax workload set with the PlugSched deployment measurement style: run Nginx, Redis, MySQL, and Memcached while live-replacing an optimized module, then measure both steady-state speedup and update-time disturbance.

Discussion. PlugPGO's central trade-off is performance versus deployability. Static PGO can optimize the whole kernel image and may produce better global layout. Dynamic module replacement restricts the optimization boundary and may introduce small overhead through redirection or conservative code placement. However, a slightly smaller speedup that can be deployed online may be more valuable than a larger speedup that is operationally expensive. In data centers, the best optimization is not simply the fastest binary; it is the fastest binary that can be safely deployed, monitored, and rolled back.

The method also suggests a practical research agenda. The first step is to identify kernel subsystems or hot functions where PGO optimization is useful and data compatibility is stable. The second step is to build a profile pipeline that can generate optimized modules automatically. The third step is to combine live replacement with performance monitoring so that updates can be rolled out gradually. The final step is to decide when a profile has become stale and whether a universal profile, application-specific profile, or hybrid profile should be used.

From an implementation perspective, the most important engineering task is to keep the optimized module tied to the exact kernel environment that produced it. The module should record the kernel version, configuration, compiler version, profile source, and symbol interface that were used during compilation. This metadata is not decorative. It allows the deployment tool to reject an optimized module when the running kernel has changed in a way that could invalidate the profile or the binary interface. Static PGO can hide this requirement because the entire optimized kernel is booted as one artifact. A hot-upgradable module must make compatibility explicit.

The second implementation task is to define what success means before deployment. PlugPGO should not install a module simply because it was compiled with a profile. It should define expected metrics, such as pipe throughput, swap performance, web-server requests per second, CPU cycles per request, or tail latency. Those metrics should be measured before and after the update. If the module does not improve the chosen target, or if it improves throughput while harming latency, rollback should be automatic. In this sense, PlugPGO is as much an observability problem as a compilation problem.

The final implementation task is to keep the first version deliberately narrow. A small module with stable data structures is easier to validate than a large subsystem with many implicit dependencies. Starting narrow also makes the performance loss relative to static PGO easier to understand: if the dynamic module is slower, the cause may be boundary overhead, missing cross-module optimization, or profile mismatch. Once those costs are measured, the boundary can be expanded carefully.

Chapter 7

Synthesis and Conclusion

7.1 Evolution and Comparative Synthesis

The four systems studied in this thesis form a coherent progression. AutoFDO proves that sampling-based feedback-directed optimization can be made practical in production-scale user-space services. Tarax moves the idea into kernel space and demonstrates that application-specific Linux kernels can accelerate real server workloads. One Profile Fits All weakens the need for per-application specialization by showing that data-center applications often use the kernel similarly enough for a merged profile to work. PlugSched solves a different but complementary problem: it shows that a tightly coupled Linux subsystem can be modularized and updated live with very short downtime. As shown in Figure 7.1, the thesis therefore treats PlugPGO as a merge point between kernel PGO, profile reuse, and live modular deployment.

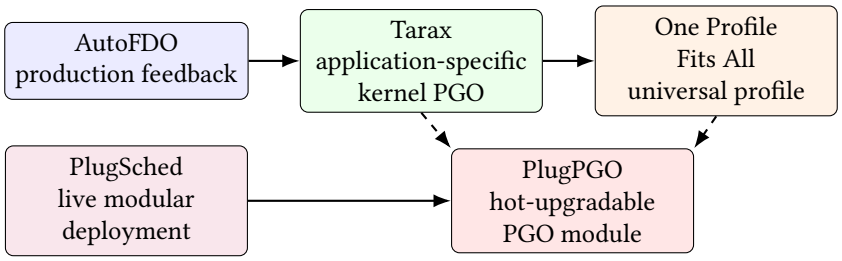


Figure 7.1: Evolution roadmap of kernel PGO for data centers.

PlugPGO is the natural synthesis of those lines. It accepts Tarax's premise that profile feedback can improve kernel code, the One Profile Fits All premise that profiles can sometimes be shared across applications, and PlugSched's premise that kernel components can be updated online. The resulting vision is an automated data-center operating system that collects profiles, builds optimized modules, deploys them live, monitors their effect, and rolls them back when necessary.

7.2 Comparative Summary

Table 7.1 summarizes the role of each system in that argument. The table is intentionally organized by goal, mechanism, strength, and limitation so that the reader can see why PlugPGO is framed as a synthesis rather than as a replacement for AutoFDO, Tarax, One Profile Fits All, or PlugSched.

Table 7.1: Comparative summary of the systems discussed in this thesis.

System	Main goal	Key mechanism	Strength	Limitation
AutoFDO	Production-scale user-space feedback optimization	Hardware sampling and binary-to-source profile conversion	Avoids intrusive instrumentation in production	User-space focus; does not solve kernel deployment
Tarax	Application-specific Linux kernels	Kernel instrumentation, profile collection, GCC-guided recompilation	8.8% average speedup across six applications	Per-application profiles and whole-kernel deployment
One Profile Fits All	General kernel PGO across data-center workloads	Similarity analysis and equal-weight merged universal profile	8.02% average speedup with one profile	Static optimized kernel artifact remains
PlugSched	Live update for Linux scheduler	Modularization, boundary analysis, function redirection, data rebuild	2.1–2.6 ms upgrade downtime and rollback support	Designed for scheduler update, not compiler PGO
PlugPGO	Hot-upgradable PGO modules	PGO-optimized module plus PlugSched-style redirection and delayed release	Combines performance optimization with online deployment	May lose part of static PGO benefit; needs conservative module boundaries

The comparison shows that no single system solves the whole problem alone. AutoFDO solves profile collection at scale, Tarax solves kernel PGO feasibility, One Profile Fits All solves profile reuse, and PlugSched solves live deployment. PlugPGO's contribution is to place these pieces in one deployment-oriented design space.

7.3 End-to-End Optimization Pipeline

The systems can also be understood as stages of a unified pipeline. The first stage is observation. A production server or benchmark environment collects profile information from real execution. AutoFDO shows how this can be done with sampled hardware counters in user space, while Tarax shows how kernel instrumentation can collect data for Linux. The second stage is interpretation. The raw profile must be mapped to compiler decisions and, in the universal-profile case, compared or merged across applications. One Profile Fits All contributes the similarity analysis needed for this stage.

The third stage is generation. The compiler uses the profile to produce optimized code. For Tarax, the output is a whole kernel image. For PlugPGO, the desired output is an optimized module or component. The fourth stage is deployment. PlugSched contributes the live-update mechanism that makes deployment compatible with high availability. The fifth

stage is monitoring. After deployment, the system must measure whether the optimization actually improves the workload and whether any regressions appear. This monitoring stage is implicit in the presentation's ideal feedback loop and should be made explicit in a production design.

This pipeline view clarifies why kernel PGO should be treated as an operating-system research problem. The compiler alone only handles generation. The operating system and data-center platform must handle observation, deployment, monitoring, and rollback. A successful design must therefore cross the boundary between compiler optimization, kernel engineering, and production operations.

7.4 Trade-Offs Across Performance, Generality, and Deployability

The thesis can be summarized by three axes: performance, generality, and deployability. Tarax is strongest on performance because it builds a profile-guided kernel for each application. Its weakness is generality and deployment cost. One Profile Fits All improves generality by showing that one profile can benefit multiple data-center applications. Its weakness is that the optimized artifact is still a static kernel. PlugSched is strongest on deployability because it demonstrates live replacement with millisecond-level downtime. Its weakness is that it is not itself a PGO optimization.

PlugPGO tries to occupy the middle of the three axes. It gives up some whole-kernel PGO freedom in exchange for live deployment. It can use either application-specific or universal profiles. It can be rolled out incrementally. The cost is that module boundaries must be chosen carefully and performance may be lower than static PGO. This trade-off is reasonable for data centers because operational risk is part of the optimization objective.

The three-axis view also helps avoid an overly narrow conclusion. It would be wrong to say that universal profiles replace application-specific profiles, or that live modules replace whole-kernel builds. Different workloads and organizations may choose different points. A stable internal service might use a static application-specific PGO kernel. A general cloud host might use a universal profile. A latency-sensitive fleet with frequent kernel updates might prefer PlugPGO. The contribution of this thesis is to show how these choices relate to one another.

7.5 Open Challenges

The first open challenge is profile freshness. A profile collected from yesterday's workload may not represent today's traffic mix, kernel version, or hardware behavior. AutoFDO studies stale profiles in user space, but kernel PGO must additionally account for kernel configuration and subsystem boundaries. A production PlugPGO pipeline would need policy rules for when to reuse, refresh, merge, or discard profiles.

The second challenge is safety of module boundaries. PlugSched succeeds because it carefully analyzes the scheduler boundary and data types. PlugPGO must do the same for any optimized module. Code-layout optimization is safe only if external interfaces, data ownership, stack behavior, and rollback semantics remain stable. As the optimized target becomes larger, the safety proof becomes harder.

The third challenge is performance attribution. If an optimized module improves a mi-

crobenchmark but not a real service, the pipeline should detect that and avoid rollout. Conversely, if a universal profile improves average throughput but hurts a latency-critical application, the system needs workload-aware policy. Kernel PGO should therefore be integrated with observability rather than treated as an offline compilation trick.

Future directions. One future direction is automated profile classification. Instead of maintaining only one universal profile, a data center could cluster services by kernel behavior and build a small set of profile classes. Web-serving, in-memory caching, storage, and database workloads might share some hot paths while diverging on others. Profile classes would preserve more specialization than a single universal profile while avoiding the management burden of one profile per service.

Another direction is profile-aware live rollout. A PlugPGO system could deploy an optimized module to a small number of machines, compare performance counters and application metrics against a control group, and then decide whether to expand rollout. This would bring kernel PGO closer to modern service deployment practices. The same system could roll back automatically if tail latency increases, if error rates rise, or if the measured speedup fails to justify the update.

A third direction is safer compiler integration. Kernel PGO needs compiler support that understands kernel constraints. The compiler should expose which transformations were performed, which functions changed layout, and whether any transformation might affect live-update compatibility. Such metadata would help a PlugPGO deployment tool decide whether a module is safe to replace online. This would connect the compiler's internal optimization decisions to the operating system's deployment policy.

Finally, kernel PGO should be studied under heterogeneous data-center hardware. Modern servers increasingly include accelerators, SmartNICs, and specialized storage devices. These components change the balance of kernel execution. A workload that previously spent many cycles in the network stack may shift work to a SmartNIC; another workload may spend more time in memory-management paths because accelerator buffers must be pinned or mapped. The profile pipeline must evolve with this hardware context.

7.6 Recommendations, Limitations, and Conclusion

The first practical recommendation is to treat profile collection as part of normal observability. A data center that wants to use kernel PGO should not collect profiles only for occasional experiments. Profiles should be connected to workload identity, kernel version, hardware class, and time. This metadata makes it possible to decide whether a profile is still trustworthy. It also makes optimization reproducible: when a PGO module helps or hurts a workload, operators can trace which profile and compiler configuration produced it.

The second recommendation is to begin with conservative module boundaries. A first PlugPGO implementation should not attempt to replace every hot path in the kernel. It should target code whose interfaces are stable, whose data layout does not change, and whose execution can be redirected safely. A conservative boundary may produce a smaller speedup than a whole-kernel Tarax image, but it is easier to validate and roll back. Once the mechanism is trusted, larger components can be studied.

The third recommendation is to separate performance validation from deployment validation. Performance validation asks whether the optimized code improves steady-state throughput, latency, or CPU efficiency. Deployment validation asks whether the update itself causes unacceptable disturbance. A PlugPGO module must pass both tests. A module with excellent

steady-state performance but a dangerous update path is not production ready. Conversely, a perfectly safe update that produces no measurable benefit is not worth deploying.

The fourth recommendation is to preserve a simple fallback path. The original kernel code should remain available until the optimized module has been accepted. Rollback should be automatic when monitoring detects a regression. This recommendation follows from PlugSched's design, but it is even more important for PGO because PGO is probabilistic and workload-dependent. The optimized path is a bet that past behavior predicts future behavior. A safe system must be able to withdraw that bet quickly.

The final recommendation is to evaluate benefit at fleet scale rather than only at microbenchmark scale. A 2% speedup on a widely deployed service can save substantial resources, while a larger speedup on a small or unstable workload may not justify additional kernel variants. The right policy therefore depends on fleet composition, service-level objectives, and operational risk. Kernel PGO should be integrated with the same rollout discipline used for application changes: canaries, comparison groups, metrics, and rollback.

These recommendations turn the thesis conclusion into an operational workflow. First, collect profiles continuously enough that they represent real services rather than a single artificial benchmark run. Second, build optimized code in a reproducible way so that every module can be traced back to a profile and source version. Third, deploy the optimized code through a live-update mechanism that keeps the old version available. Fourth, measure the effect on service-level metrics and decide whether to commit or roll back. This workflow is broader than any one paper, but each of the cited systems contributes one necessary piece.

The most important design lesson is that performance and safety should not be treated as separate phases. A profile that improves average throughput but cannot be rolled out safely is incomplete. A live-update mechanism that can replace code but has no performance objective is also incomplete. The data-center kernel should instead be designed as an adaptive platform: it observes workload behavior, compiles optimized code, deploys cautiously, and keeps monitoring after deployment. This is the practical meaning of combining Tarax, One Profile Fits All, PlugSched, and PlugPGO.

Limitations of this thesis. This thesis is a synthesis based on the presentation and the cited research papers, not a complete implementation report for a production PlugPGO system. The Tarax, One Profile Fits All, and PlugSched results are taken from their original experimental contexts. Their workloads, kernel versions, compilers, and hardware platforms are not identical. Therefore, the numerical results should be compared as evidence for design direction rather than as a single controlled benchmark suite. The common conclusion is robust: kernel PGO can improve performance, profiles can sometimes be shared, and live component update can be fast. The exact speedup of a combined PlugPGO pipeline still requires a unified implementation and evaluation.

Another limitation is that the PlugPGO discussion focuses on module-level code replacement where data structures remain compatible. This is the safest initial design, but it restricts the optimization space. Some compiler optimizations are most effective when they can reorganize a larger portion of the kernel or inline across boundaries. Some kernel subsystems may also require state migration if a new optimized version changes internal representation. Extending PlugPGO to those cases would require reintroducing parts of PlugSched's data rebuild and stack inspection logic.

The thesis also does not claim that PGO is the only method for data-center kernel optimization. Manual tuning, eBPF-based extension, scheduler redesign, library operating systems, unikernels, and user-space networking systems all address related performance problems. The reason this thesis focuses on PGO is that it offers a conservative path: retain Linux com-

patibility, observe real workloads, and let the compiler specialize code. That path is attractive precisely because it can coexist with existing Linux operations.

Finally, the evaluation of PlugPGO in the presentation is preliminary. UnixBench pipe and swap results are useful for checking whether dynamic replacement changes performance, but they are not enough to establish fleet-level value. A full evaluation should combine real services, profile freshness experiments, canary rollout, rollback tests, and tail-latency measurement. These missing experiments define the next stage of work.

Conclusion. This thesis has argued that profile-guided optimization for the Linux kernel is a promising direction for data-center operating systems, but only if performance optimization is paired with deployability. Tarax demonstrates that application-specific kernels can produce substantial speedup. One Profile Fits All shows that many data-center workloads share enough kernel behavior for a universal profile to be effective. PlugSched demonstrates that a complex Linux subsystem can be modularized and updated live with millisecond-level downtime. PlugPGO combines these insights by proposing hot-upgradable PGO modules.

The broader lesson is that the future data-center kernel should be adaptive. It should observe how services actually use the kernel, compile optimized code for those patterns, deploy improvements online, and roll back safely when the optimization is wrong. This closed-loop view turns the operating system from a static general-purpose artifact into a continuously optimized platform for large-scale applications.

Bibliography

[1] Jeff Arnold and M. Frans Kaashoek. Ksplice: Automatic rebootless kernel updates. In *Proceedings of the 4th ACM European Conference on Computer Systems*, pages 187–198. ACM, 2009. doi: 10.1145/1519065.1519085.

[2] Adam Belay, George Prekas, Mia Primorac, Ana Klimovic, Samuel Grossman, Christos Kozyrakis, and Edouard Bugnion. IX: A protected dataplane operating system for high throughput and low latency. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation*, pages 49–65. USENIX Association, 2014.

[3] Dehao Chen, David Xinliang Li, and Tipp Moseley. AutoFDO: Automatic feedback-directed optimization for warehouse-scale applications. In *Proceedings of the 2016 International Symposium on Code Generation and Optimization*, pages 12–23. ACM, 2016. doi: 10.1145/2854038.2854044.

[4] Dawson R. Engler, M. Frans Kaashoek, and James O'Toole. Exokernel: An operating system architecture for application-level resource management. In *Proceedings of the Fifteenth ACM Symposium on Operating Systems Principles*, pages 251–266. ACM, 1995. doi: 10.1145/224056.224076.

[5] Linux Kernel Community. Livepatch: Live patching of the linux kernel. `https://docs.kernel.org/livepatch/livepatch.html`, 2026. Referenced for background on function-level kernel live patching.

[6] Teng Ma, Shanpei Chen, Yihao Wu, Erwei Deng, Zhuo Song, Quan Chen, and Minyi Guo. Efficient scheduler live update for linux kernel with modularization. In *Proceedings of the 28th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*, pages 191–206. ACM, 2023. doi: 10.1145/3582016.3582054.

[7] Anil Madhavapeddy, Richard Mortier, Charalampos Rotsos, David Scott, Balraj Singh, Thomas Gazagnaire, Steven Smith, Steven Hand, and Jon Crowcroft. Unikernels: Library operating systems for the cloud. In *Proceedings of the Eighteenth International Conference on Architectural Support for Programming Languages and Operating Systems*, pages 461–472. ACM, 2013. doi: 10.1145/2451116.2451167.

[8] Maksim Panchenko, Rafael Auler, Bill Nell, and Guilherme Ottoni. BOLT: A practical binary optimizer for data centers and beyond. In *Proceedings of the 2019 IEEE/ACM International Symposium on Code Generation and Optimization*, pages 2–14. IEEE, 2019.

[9] Simon Peter, Jialin Li, Irene Zhang, Dan R. K. Ports, Doug Woos, Arvind Krishnamurthy, Thomas Anderson, and Timothy Roscoe. Arrakis: The operating system is the control

plane. In *Proceedings of the 11th USENIX Symposium on Operating Systems Design and Implementation*, pages 1–16. USENIX Association, 2014.

[10] Karl Pettis and Robert C. Hansen. Profile guided code positioning. In *Proceedings of the ACM SIGPLAN 1990 Conference on Programming Language Design and Implementation*, pages 16–27. ACM, 1990. doi: 10.1145/93542.93550.

[11] Red Hat. kpatch: Dynamic kernel patching. `https://github.com/dynup/kpatch`, 2026. Referenced for background on Linux live patching.

[12] Barret Rhoden et al. ghOSt: Fast and flexible user-space delegation of linux scheduling. In *Proceedings of the ACM SIGOPS 28th Symposium on Operating Systems Principles*, pages 588–604. ACM, 2021. doi: 10.1145/3477132.3483542.

[13] Muhammed Ugur, Cheng Jiang, Alex Erf, Tanvir Ahmed Khan, and Baris Kasikci. One profile fits all: Profile-guided linux kernel optimizations for data center applications. *ACM SIGOPS Operating Systems Review*, 56(1):26–33, 2022. doi: 10.1145/3544497.3544502.

[14] Pengfei Yuan, Yao Guo, Lu Zhang, Xiangqun Chen, and Hong Mei. Building application-specific operating systems: A profile-guided approach. *Science China Information Sciences*, 61(9):092102:1–092102:17, 2018. doi: 10.1007/s11432-017-9418-9.